Spam Detection Using Traditional Machine Learning, Deep Learning and Transformer Models

Project Overview

Spam messages and emails are one of the most common forms of unwanted digital communication. These messages often contain advertisements, phishing attempts, scams, fraudulent offers, or misleading information intended to deceive users.

The objective of this project is to build an automated spam detection system capable of classifying incoming messages as either Spam or Ham (Legitimate).

Rather than relying on a single modelling approach, our team aims to compare multiple Natural Language Processing (NLP) techniques across three major categories:

- Traditional Machine Learning
- Deep Learning
- Transformer-based Models

The final goal is to identify which approach provides the best balance between performance, generalization capability, and computational efficiency.

The project will conclude with a deployed Streamlit application that demonstrates real-time spam classification.

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import re
import string
import nltk
import json
import matplotlib.pyplot as plt
import os

os.makedirs(
    "../results/figures",
    exist_ok=True
)

from sklearn.preprocessing import LabelEncoder
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split

nltk.download("wordnet")
nltk.download("punkt")
nltk.download("stopwords")
print("Imports Done")
```

```
Imports Done
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\rstar\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\rstar\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\rstar\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Dataset Loading and Initial Inspection

In this section, the SMS Spam Collection dataset is loaded and inspected.

The original dataset contains labelled SMS messages belonging to two categories:

- Ham (Legitimate Message)
- Spam (Unwanted Message)

In [2]:
```
df = pd.read_csv(
    "../data/raw/spam.csv",
    encoding="latin-1"
)

df.drop(
    columns=[
        'Unnamed: 2',
        'Unnamed: 3',
        'Unnamed: 4'
    ],
    inplace=True
)

df.rename(
    columns={
        'v1':'label',
        'v2':'text'
    },
    inplace=True
)

df.head()
```

Out[2]:

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Data Quality Assessment

Real-world datasets frequently contain duplicate records that may bias model training and evaluation.

To ensure fair learning and reliable performance measurement, duplicate messages are identified and removed from the dataset.

In [3]:
```
print(df.shape)

print("\nMissing Values")
print(df.isnull().sum())

print("\nDuplicates")
print(df.duplicated().sum())
```

```
(5572, 2)

Missing Values
label    0
text     0
dtype: int64

Duplicates
403
```

In [4]:
```
df = df.drop_duplicates()

print(df.shape)
```

```
(5169, 2)
```

Exploratory Data Analysis (EDA)

Before applying any NLP techniques, we perform exploratory analysis to better understand the characteristics of the dataset.

The following analyses are performed:

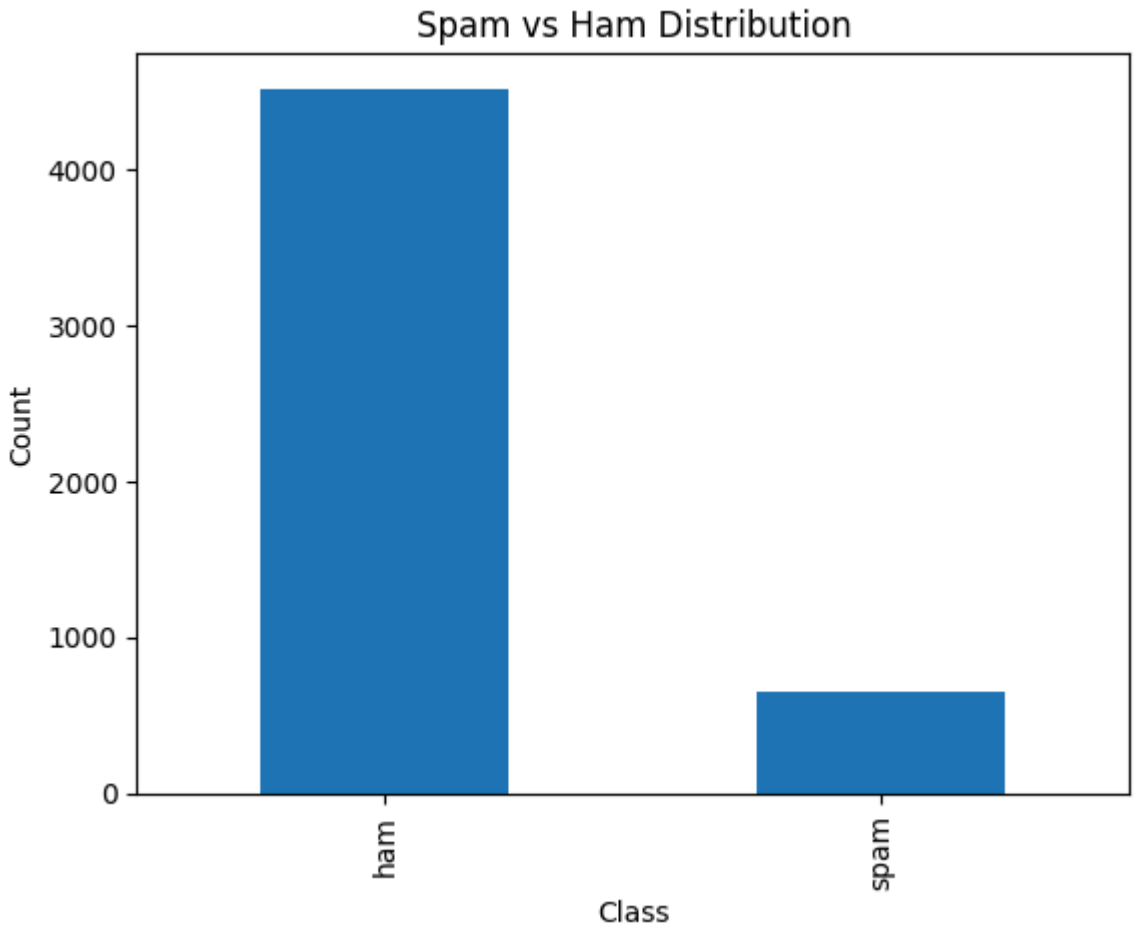
- Distribution of Spam and Ham messages
- Message length statistics
- Character count analysis
- Word count analysis

These insights help us understand class balance and message characteristics that may influence model behaviour.

In [5]:
```
print(df["label"].value_counts())

print()

print(
    df["label"].value_counts(normalize=True)*100
)
```

```
label
ham     4516
spam     653
Name: count, dtype: int64

label
ham     87.366996
spam    12.633004
Name: proportion, dtype: float64
```

In [6]:
```
df["label"].value_counts().plot(
    kind="bar"
)

plt.title(
    "Spam vs Ham Distribution"
)

plt.xlabel("Class")
plt.ylabel("Count")
plt.savefig(
    "../results/figures/class_distribution.png",
    bbox_inches="tight"
)
plt.show()
```



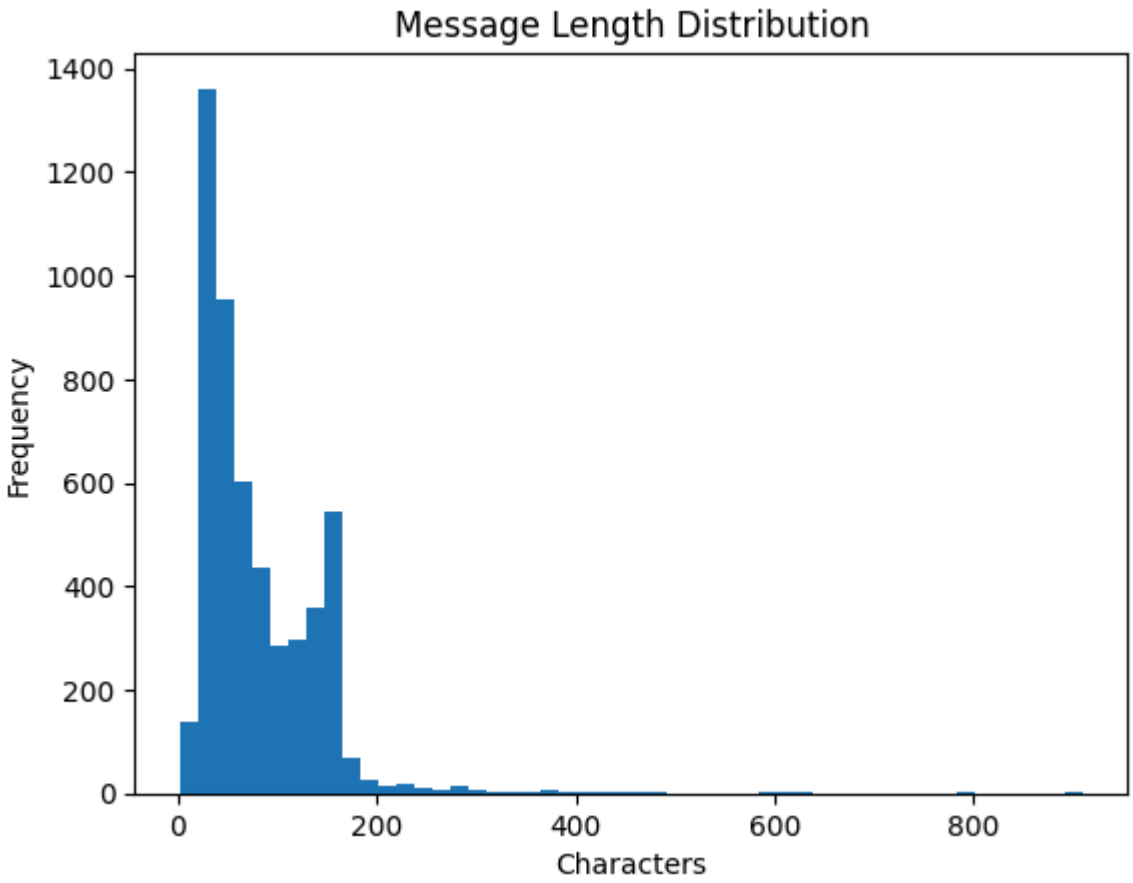
In [7]:
```
df["char_count"] = df["text"].apply(len)

df["word_count"] = df["text"].apply(
    lambda x: len(x.split())
)
```

In [8]:
```
print(
    df.groupby("label")[
        ["char_count","word_count"]
    ].mean()
)
```

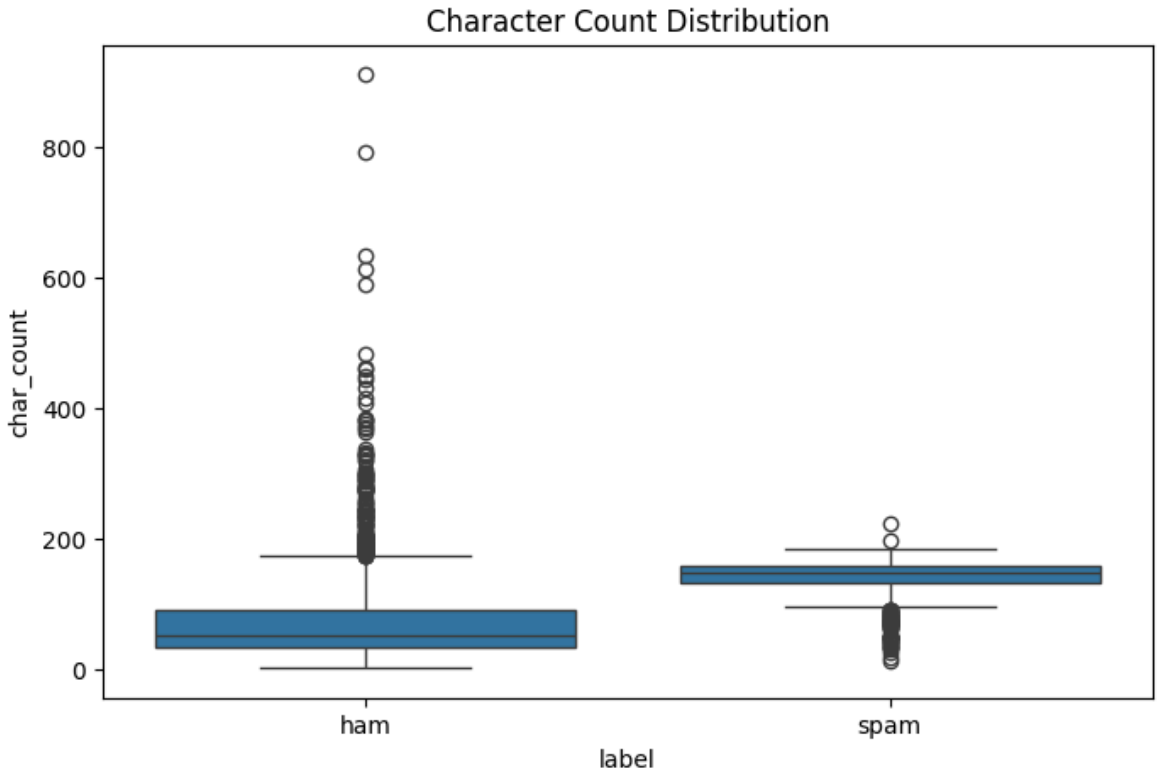
```
       char_count  word_count
label
ham     70.459256   14.134632
spam   137.891271   23.681470
```

In [9]:
```
plt.hist(
    df["char_count"],
    bins=50
)

plt.title(
    "Message Length Distribution"
)

plt.xlabel("Characters")
plt.ylabel("Frequency")
plt.savefig(
    "../results/figures/message_length_distribution.png",
    bbox_inches="tight"
```

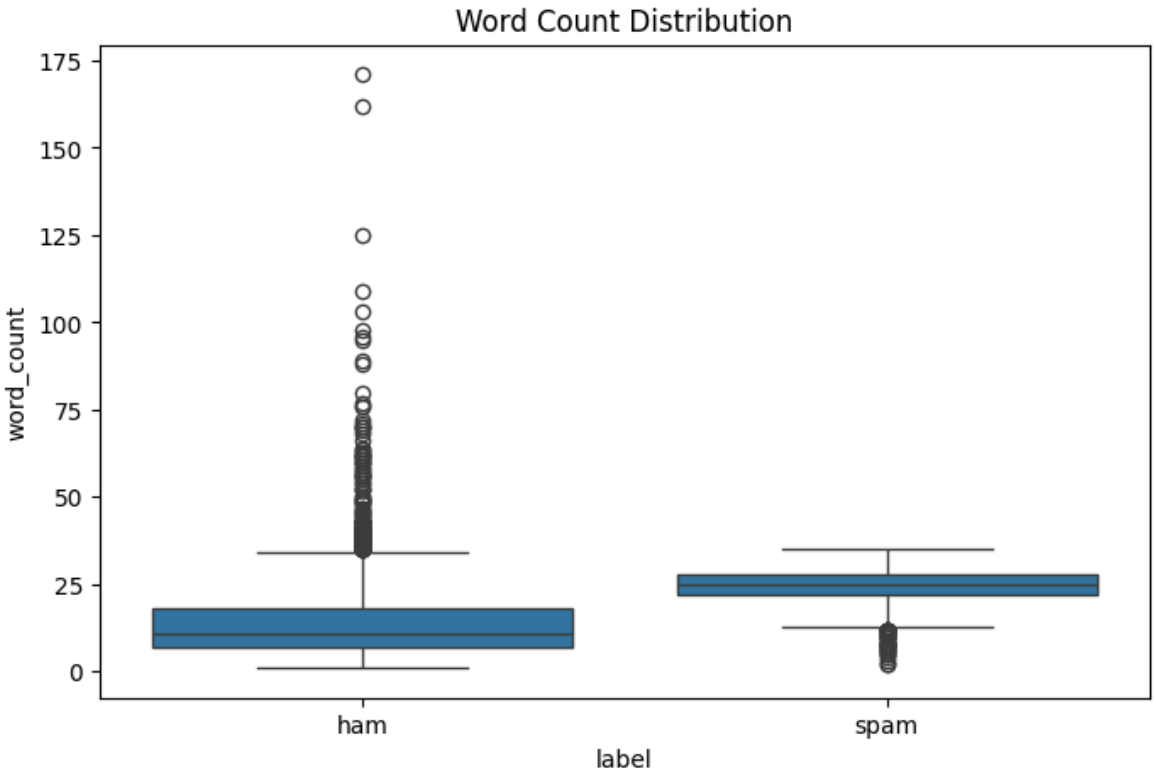
```
)
plt.show()
```



In [10]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="label",
    y="char_count"
)

plt.title(
    "Character Count Distribution"
)

plt.savefig(
    "../results/figures/char_count_boxplot.png",
    bbox_inches="tight"
)

plt.show()
```



In [11]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="label",
    y="word_count"
)

plt.title(
    "Word Count Distribution"
)

plt.savefig(
    "../results/figures/word_count_boxplot.png",
    bbox_inches="tight"
)

plt.show()
```



Text Preprocessing

Raw text cannot be directly used by machine learning algorithms.

Therefore, a preprocessing pipeline is applied to standardize the text and reduce noise.

The preprocessing steps include:

1. Lowercasing
2. URL Removal
3. Number Removal
4. Punctuation Removal
5. Stopword Removal
6. Lemmatization

The output of this stage is a cleaned text representation that will be used during feature extraction and model training.

In [12]:
```
stop_words = set(
    stopwords.words("english")
)

lemmatizer = WordNetLemmatizer()

def clean_text(text):

    text = text.lower()

    text = re.sub(
        r"http\S+|www\S+",
        " ",
```

```
        text
    )

    text = re.sub(
        r"\d+",
        " ",
        text
    )

    text = text.translate(
        str.maketrans(
            "",
            "",
            string.punctuation
        )
    )

    tokens = nltk.word_tokenize(text)

    tokens = [
        word
        for word in tokens
        if word not in stop_words
    ]

    tokens = [
    lemmatizer.lemmatize(word)
    for word in tokens
    ]

    return " ".join(tokens)
```

In [13]:

```
df["clean_text"] = df["text"].apply(
    clean_text
)
df["clean_text"] = (
    df["clean_text"]
    .fillna("")
    .astype(str)
)
df = df[
    df["clean_text"].str.strip() != ""
]
df[
    ["text","clean_text"]
].head()
```

Out[13]:

	text	clean_text
0	Go until jurong point, crazy.. Available only ...	go jurong point crazy available bugis n great ...
1	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	Free entry in 2 a wkly comp to win FA Cup fina...	free entry wkly comp win fa cup final tkts st ...
3	U dun say so early hor... U c already then say...	u dun say early hor u c already say
4	Nah I don't think he goes to usf, he lives aro...	nah dont think go usf life around though

In [14]:
```
print(
    "Remaining NaNs:",
    df["clean_text"].isna().sum()
)

print(
    "Empty strings:",
    (df["clean_text"].str.strip() == "").sum()
)
```

```
Remaining NaNs: 0
Empty strings: 0
```

In [15]:
```
summary = pd.DataFrame({

    "Metric":[
        "Total Messages",
        "Spam Messages",
        "Ham Messages"
    ],

    "Value":[
        len(df),
        (df["label"]=="spam").sum(),
        (df["label"]=="ham").sum()
    ]
})

print(summary)

summary.to_csv(
    "../results/dataset_summary.csv",
    index=False
)
```

```
          Metric  Value
0  Total Messages   5163
1   Spam Messages    653
2    Ham Messages   4510
```

Label Encoding

Machine learning algorithms require numerical target values.

The original class labels are therefore converted into numerical form:

- Ham → 0
- Spam → 1

This encoded representation will be used consistently throughout the remainder of the project.

In [16]:
```
encoder = LabelEncoder()

df["target"] = encoder.fit_transform(
    df["label"]
)

print(df[["label","target"]].head())
```

```
  label  target
0   ham       0
1   ham       0
2  spam       1
3   ham       0
4   ham       0
```

Dataset Splitting Strategy

To ensure fair and reproducible evaluation, the dataset is divided into three subsets:

- Training Set (70%)
- Validation Set (15%)
- Test Set (15%)

The split is performed using stratified sampling to preserve the original class distribution across all subsets.

These saved splits will be reused by every subsequent notebook in the project, ensuring that all machine learning, deep learning, and transformer models are evaluated on exactly the same data.

In [17]:
```
train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    stratify=df["label"],
    random_state=42
)
```

In [18]:
```
val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df["label"],
    random_state=42
)
```

In [19]:
```
print("Train:", train_df.shape)
print("Validation:", val_df.shape)
```

```
print("Test:", test_df.shape)
```

```
Train: (3614, 6)
Validation: (774, 6)
Test: (775, 6)
```

In [20]:
```
split_stats = pd.DataFrame({

    "Dataset":[
        "Train",
        "Validation",
        "Test"
    ],

    "Rows":[
        len(train_df),
        len(val_df),
        len(test_df)
    ]
})
split_stats.to_csv(
    "../results/split_statistics.csv",
    index=False
)
```

In [21]:
```
print(
    train_df["label"]
    .value_counts(normalize=True)
)

print(
    val_df["label"]
    .value_counts(normalize=True)
)

print(
    test_df["label"]
    .value_counts(normalize=True)
)
```

```
label
ham     0.873547
spam    0.126453
Name: proportion, dtype: float64
label
ham     0.873385
spam    0.126615
Name: proportion, dtype: float64
label
ham     0.873548
spam    0.126452
Name: proportion, dtype: float64
```

Data Persistence

The processed dataset and generated splits are saved for future use.

The saved files will serve as the common input source for:

- Traditional Machine Learning Models
- Deep Learning Models
- Transformer-based Models

This guarantees consistency across all experiments and prevents accidental changes to the training and testing data.

In [22]:
```
import os

os.makedirs(
    "../data/processed",
    exist_ok=True
)
```

In [23]:
```
print(train_df.columns)

print(val_df.columns)

print(test_df.columns)
```

```
Index(['label', 'text', 'char_count', 'word_count', 'clean_text', 'target'], dtyp
e='object')
Index(['label', 'text', 'char_count', 'word_count', 'clean_text', 'target'], dtyp
e='object')
Index(['label', 'text', 'char_count', 'word_count', 'clean_text', 'target'], dtyp
e='object')
```

In [24]:
```
train_df.to_csv(
    "../data/processed/train.csv",
    index=False
)

val_df.to_csv(
    "../data/processed/val.csv",
    index=False
)

test_df.to_csv(
    "../data/processed/test.csv",
    index=False
)
```

In [25]:
```
metadata = {

    "dataset_shape": list(df.shape),

    "features": list(df.columns),

    "random_state": 42,

    "split_ratio": {
        "train": 0.70,
        "validation": 0.15,
        "test": 0.15
    },

    "preprocessing": [
        "lowercase",
```

```
        "url_removal",
        "number_removal",
        "punctuation_removal",
        "stopword_removal",
        "lemmatization"
    ]
}

with open(
    "../data/processed/preprocessing_metadata.json",
    "w"
) as f:
    json.dump(
        metadata,
        f,
        indent=4
    )
print("Metadata Saved")

print(
    "../data/processed/train.csv"
)
print(
    "../data/processed/val.csv"
)
print(
    "../data/processed/test.csv"
)
```

```
Metadata Saved
../data/processed/train.csv
../data/processed/val.csv
../data/processed/test.csv
```

Conclusion

In this notebook, we completed the data preparation phase of the spam detection project.

The dataset was successfully loaded, cleaned, analysed, and transformed into a format suitable for machine learning workflows. Duplicate records were removed, exploratory analysis was performed, and a comprehensive preprocessing pipeline was applied to standardize the text data.

The target labels were encoded and the dataset was split into training, validation, and testing subsets using a stratified strategy. These splits were then saved for reuse throughout the project.

The outputs generated in this notebook will be used as the foundation for all subsequent experiments.

Next Steps:

1. Traditional Machine Learning Models

- Naive Bayes
- Logistic Regression
- Support Vector Machine (SVM)

2. Deep Learning Models

- LSTM
- Bidirectional LSTM

3. Transformer Models

- DistilBERT
- BERT

Finally, all approaches will be compared using common evaluation metrics such as Accuracy, Precision, Recall, F1 Score, and ROC-AUC. The best-performing model will then be integrated into a Streamlit-based web application for demonstration purposes.

Traditional Machine Learning for Spam Detection

Overview

In this notebook, we implement and evaluate traditional machine learning approaches for spam detection using the preprocessed dataset generated in the previous phase.

Traditional machine learning remains an important baseline for text classification tasks due to its simplicity, efficiency, and interpretability. These models typically rely on feature engineering techniques such as TF-IDF to transform textual data into numerical representations suitable for machine learning algorithms.

The primary objective of this notebook is to compare multiple traditional machine learning classifiers and establish baseline performance before moving to more advanced Deep Learning and Transformer-based approaches.

The models evaluated in this notebook are:

- Multinomial Naive Bayes
- Logistic Regression
- Linear Support Vector Machine (SVM)

All models use the same train, validation, and test splits generated during data preparation to ensure fair comparison throughout the project.

In [1]:
```
import pandas as pd
import numpy as np
import joblib
import os

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
import os

os.makedirs(
    "../results/figures",
    exist_ok=True
)
```

In [3]:
```
train_df = pd.read_csv("../data/processed/train.csv")
val_df = pd.read_csv("../data/processed/val.csv")
test_df = pd.read_csv("../data/processed/test.csv")

print(train_df.shape)
print(val_df.shape)
print(test_df.shape)
```

```
(3614, 6)
(774, 6)
(775, 6)
```

In [4]:
```
X_train = train_df["clean_text"]

X_val = val_df["clean_text"]

X_test = test_df["clean_text"]

y_train = train_df["target"]

y_val = val_df["target"]

y_test = test_df["target"]
```

Feature Engineering using TF-IDF

Machine learning algorithms cannot directly process raw text.

To convert textual messages into numerical features, Term Frequency – Inverse Document Frequency (TF-IDF) vectorization is applied.

TF-IDF assigns greater importance to words that are informative within a document while reducing the influence of extremely common words that appear frequently across the entire dataset.

The resulting feature matrix will serve as the input representation for all traditional machine learning models implemented in this notebook.

In [5]:
```
tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1,2)
)

X_train_tfidf = tfidf.fit_transform(
    X_train
)

X_val_tfidf = tfidf.transform(
    X_val
```

```
)

X_test_tfidf = tfidf.transform(
    X_test
)

print(X_train_tfidf.shape)
```

```
(3614, 5000)
```

In [6]:
```
results = []
```

In [7]:
```
def evaluate_model(
    model_name,
    y_true,
    y_pred
):

    accuracy = accuracy_score(
        y_true,
        y_pred
    )

    precision = precision_score(
        y_true,
        y_pred
    )

    recall = recall_score(
        y_true,
        y_pred
    )

    f1 = f1_score(
        y_true,
        y_pred
    )

    results.append({
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1": f1
    })

    print(f"\n{model_name}")
    print(
        classification_report(
            y_true,
            y_pred
        )
    )
```

Multinomial Naive Bayes

Multinomial Naive Bayes is one of the most widely used algorithms for text classification problems.

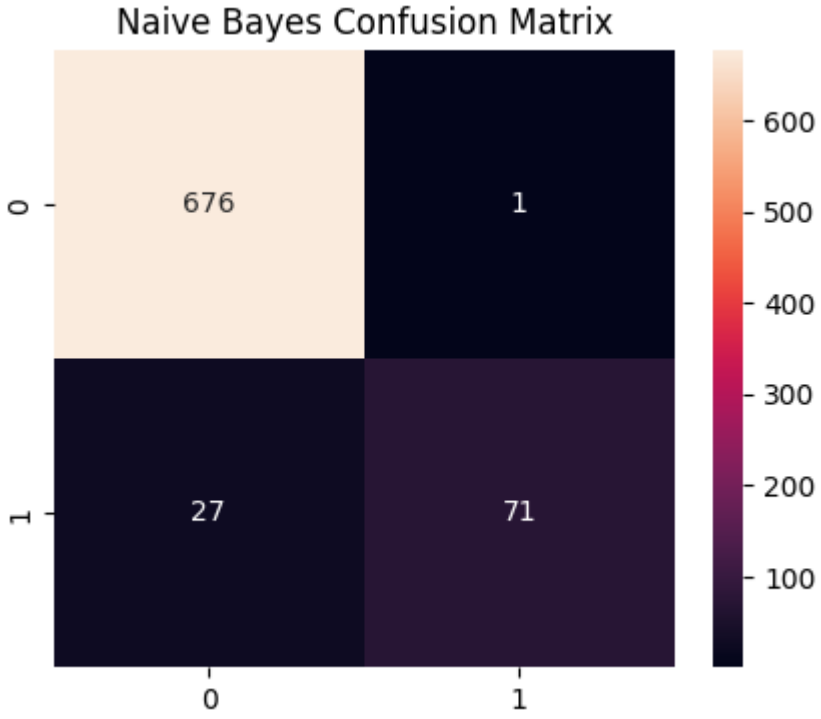
The model assumes conditional independence among features and estimates class probabilities based on word occurrence frequencies.

Despite its simplicity, Naive Bayes often performs remarkably well on spam detection tasks and serves as a strong baseline model.

In [8]:
```
nb = MultinomialNB()

nb.fit(
    X_train_tfidf,
    y_train
)

nb_pred = nb.predict(
    X_test_tfidf
)

evaluate_model(
    "Naive Bayes",
    y_test,
    nb_pred
)
```

```
Naive Bayes
              precision    recall  f1-score   support

           0       0.96      1.00      0.98       677
           1       0.99      0.72      0.84        98

    accuracy                           0.96       775
   macro avg       0.97      0.86      0.91       775
weighted avg       0.96      0.96      0.96       775
```

In [9]:
```
cm = confusion_matrix(
    y_test,
    nb_pred
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title(
    "Naive Bayes Confusion Matrix"
)
plt.savefig(
    "../results/figures/nb_confusion_matrix.png",
    bbox_inches="tight"
)
plt.show()
```



Naive Bayes Results Analysis

The Multinomial Naive Bayes classifier achieved an accuracy of approximately 96.39% and an F1 Score of 0.8353.

The confusion matrix indicates that the model correctly classified 676 ham messages and 71 spam messages. Only one legitimate message was incorrectly classified as spam, demonstrating very strong precision.

However, the model failed to identify 27 spam messages, resulting in a recall of only 72.45%. This suggests that while Naive Bayes is highly conservative when predicting spam, it misses a considerable number of actual spam messages.

For spam detection systems, missed spam messages can reduce the overall effectiveness of the classifier despite strong precision.

Logistic Regression

Logistic Regression is a linear classification algorithm that estimates class probabilities using a logistic function.

Unlike Naive Bayes, Logistic Regression learns feature weights directly from the data, allowing it to capture more discriminative relationships between words and class labels.

It is commonly used as a strong baseline for many natural language processing tasks.

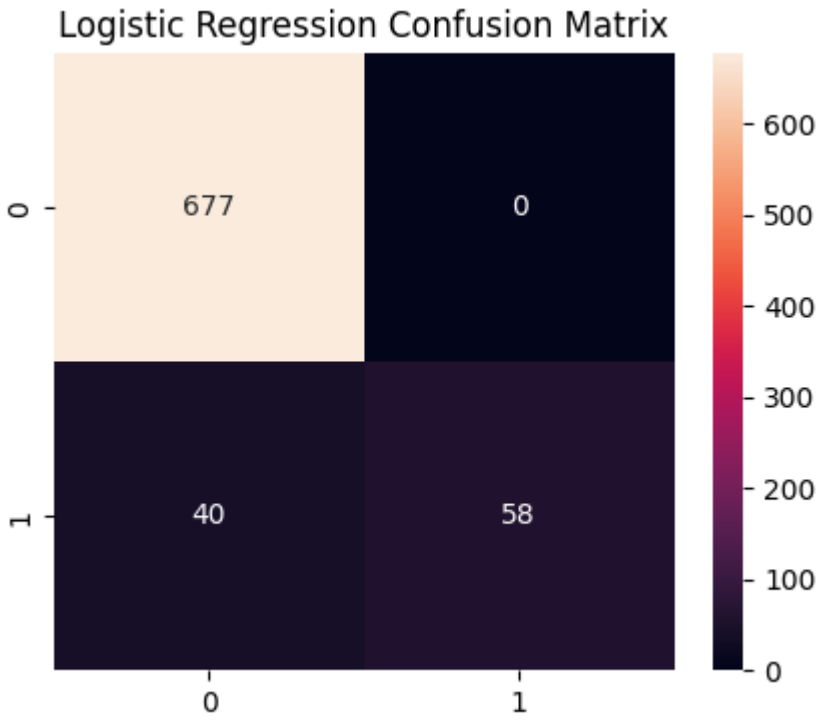
In [10]:
```
lr = LogisticRegression(
    max_iter=1000,
    random_state=42
)
```

```
lr.fit(
    X_train_tfidf,
    y_train
)

lr_pred = lr.predict(
    X_test_tfidf
)

evaluate_model(
    "Logistic Regression",
    y_test,
    lr_pred
)
```

```
Logistic Regression
              precision    recall  f1-score   support

           0       0.94      1.00      0.97       677
           1       1.00      0.59      0.74        98

    accuracy                           0.95       775
   macro avg       0.97      0.80      0.86       775
weighted avg       0.95      0.95      0.94       775
```

In [11]:

```
cm = confusion_matrix(
    y_test,
    lr_pred
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title(
    "Logistic Regression Confusion Matrix"
)
plt.savefig(
    "../results/figures/lr_confusion_matrix.png",
    bbox_inches="tight"
)
plt.show()
```



Logistic Regression Results Analysis

Logistic Regression achieved an accuracy of approximately 94.84% and an F1 Score of 0.7436.

The model produced perfect precision (100%), meaning every message predicted as spam was actually spam. No legitimate messages were incorrectly classified as spam.

However, the confusion matrix reveals that the model correctly detected only 58 spam messages while missing 40 spam instances. This resulted in a recall of only 59.18%, the lowest among all evaluated models.

Although the model is extremely cautious when identifying spam, the large number of undetected spam messages significantly reduces its practical usefulness for real-world spam filtering applications.

Linear Support Vector Machine (SVM)

Support Vector Machines are highly effective for high-dimensional text classification problems.

The Linear SVM attempts to identify an optimal decision boundary that maximizes the margin between spam and ham messages.

Due to its ability to handle sparse TF-IDF representations effectively, Linear SVM frequently achieves state-of-the-art performance among traditional machine learning methods.

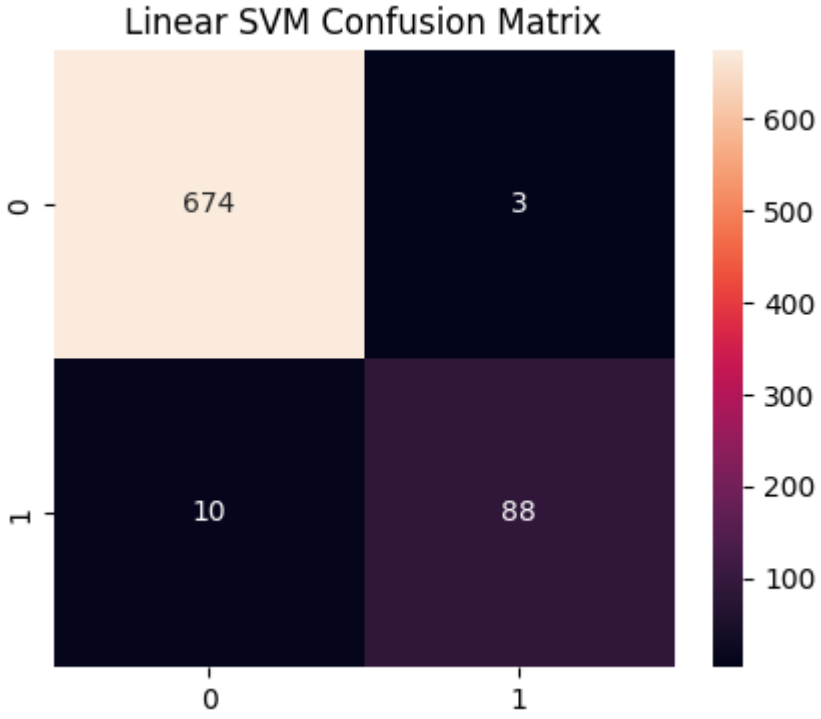
In [12]:
```
svm = LinearSVC(
    random_state=42
```

```
)

svm.fit(
    X_train_tfidf,
    y_train
)

svm_pred = svm.predict(
    X_test_tfidf
)

evaluate_model(
    "Linear SVM",
    y_test,
    svm_pred
)
```

```
Linear SVM
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       677
           1       0.97      0.90      0.93        98

    accuracy                           0.98       775
   macro avg       0.98      0.95      0.96       775
weighted avg       0.98      0.98      0.98       775
```

In [13]:

```
cm = confusion_matrix(
    y_test,
    svm_pred
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title(
    "Linear SVM Confusion Matrix"
)
plt.savefig(
    "../results/figures/svm_confusion_matrix.png",
    bbox_inches="tight"
)
plt.show()
```



Linear SVM Results Analysis

The Linear Support Vector Machine achieved the best overall performance among all traditional machine learning models evaluated.

The model obtained an accuracy of 98.32%, precision of 96.70%, recall of 89.80%, and an F1 Score of 0.9312.

The confusion matrix shows that 674 ham messages and 88 spam messages were correctly classified. Only 13 total misclassifications occurred, consisting of 3 false positives and 10 false negatives.

Compared to Naive Bayes and Logistic Regression, Linear SVM maintained a significantly better balance between precision and recall, making it more effective at detecting spam while minimizing incorrect classifications.

Model Evaluation

Each trained model is evaluated using the test dataset.

The following performance metrics are considered:

- Accuracy
- Precision
- Recall
- F1 Score

Confusion matrices are also generated to visualize the classification behaviour of each model and identify potential false positives and false negatives.

In [14]:
```
results_df = pd.DataFrame(
    results
)

results_df
```

Out[14]:

	Model	Accuracy	Precision	Recall	F1
0	Naive Bayes	0.963871	0.986111	0.724490	0.835294
1	Logistic Regression	0.948387	1.000000	0.591837	0.743590
2	Linear SVM	0.983226	0.967033	0.897959	0.931217

In [15]:
```
results_df = results_df.sort_values(
    by="F1",
    ascending=False
)

results_df
```

Out[15]:

	Model	Accuracy	Precision	Recall	F1
2	Linear SVM	0.983226	0.967033	0.897959	0.931217
0	Naive Bayes	0.963871	0.986111	0.724490	0.835294
1	Logistic Regression	0.948387	1.000000	0.591837	0.743590

Model Ranking

Based on F1 Score, the models were ranked as follows:

1. Linear SVM (0.9312)
2. Naive Bayes (0.8353)
3. Logistic Regression (0.7436)

Linear SVM outperformed the other classifiers across nearly all evaluation metrics and demonstrated the strongest ability to detect spam messages without significantly increasing false positives.

Naive Bayes performed reasonably well and maintained excellent precision, but its lower recall resulted in many spam messages remaining undetected.

Logistic Regression achieved perfect precision but suffered from the lowest recall, causing a substantial number of spam messages to be incorrectly classified as legitimate messages.

Comparative Analysis

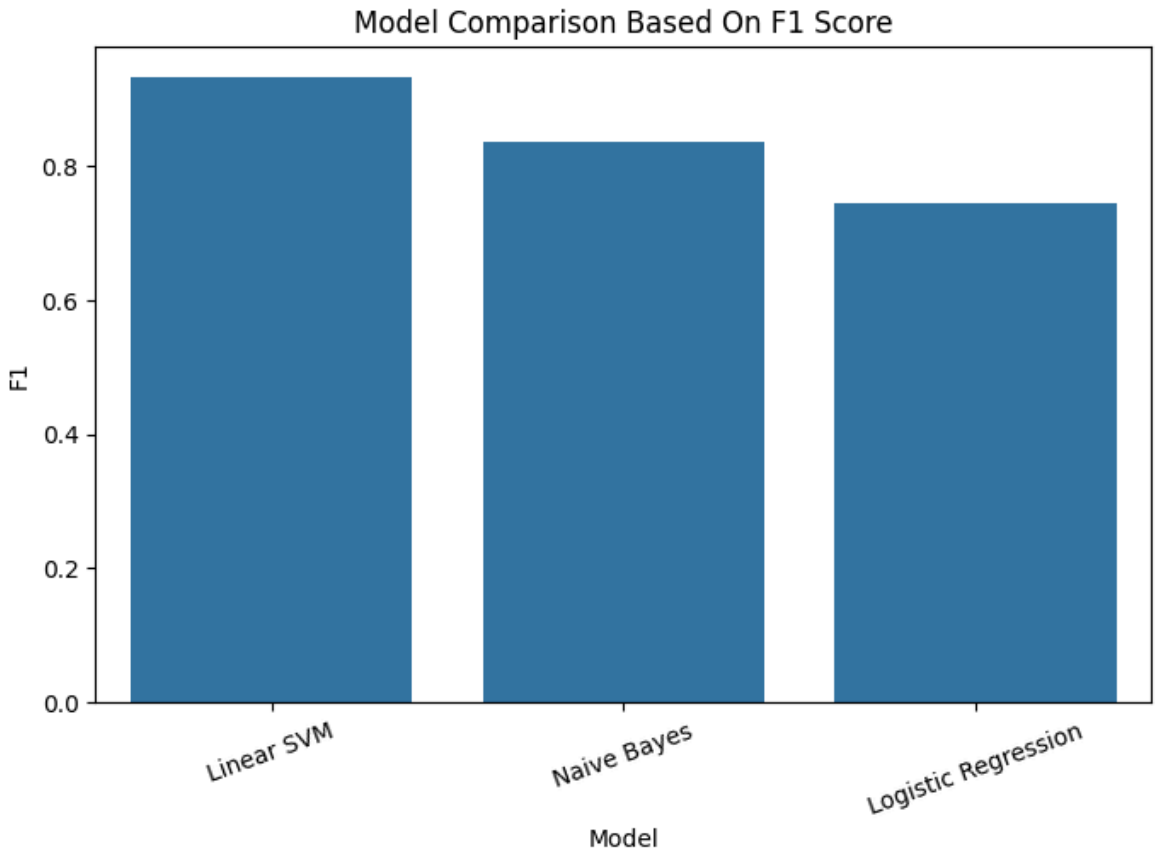
After evaluating all models individually, a comparative analysis is performed.

The objective is to identify:

- The most accurate model
- The model with the highest precision
- The model with the highest recall
- The model with the best overall F1 Score

These findings will be compared with Deep Learning and Transformer-based models in subsequent notebooks.

In [16]:
```
plt.figure(figsize=(8,5))

sns.barplot(
    data=results_df,
    x="Model",
    y="F1"
)

plt.title(
    "Model Comparison Based On F1 Score"
)

plt.xticks(rotation=20)
plt.savefig(
    "../results/figures/model_f1_comparison.png",
    bbox_inches="tight"
)
plt.show()
```



In [17]:
```
os.makedirs(
    "../models",
    exist_ok=True
)
```

```
joblib.dump(
    tfidf,
    "../models/tfidf.pkl"
)

joblib.dump(
    nb,
    "../models/naive_bayes.pkl"
)

joblib.dump(
    lr,
    "../models/logistic_regression.pkl"
)

joblib.dump(
    svm,
    "../models/linear_svm.pkl"
)
```

Out[17]:
```
['../models/linear_svm.pkl']
```

In [18]:
```
os.makedirs(
    "../results",
    exist_ok=True
)

results_df.to_csv(
    "../results/traditional_ml_results.csv",
    index=False
)

results_df

print(
    "Models Saved Successfully"
)

print(
    "Results Saved Successfully"
)
```

```
Models Saved Successfully
Results Saved Successfully
```

Conclusion

In this notebook, three traditional machine learning classifiers were developed and evaluated for the spam detection task using TF-IDF feature representations.

The experimental results demonstrated clear differences in model behaviour.

- Logistic Regression achieved perfect precision but suffered from low recall, resulting in many spam messages being missed.

- Naive Bayes produced strong precision and overall performance but still failed to identify a substantial portion of spam messages.
- Linear SVM delivered the strongest overall performance, achieving an accuracy of 98.32% and an F1 Score of 0.9312.

The confusion matrix analysis further confirmed that Linear SVM generated the fewest classification errors and provided the most balanced trade-off between precision and recall.

Based on these findings, Linear SVM is selected as the best-performing traditional machine learning model for this dataset.

The trained models, TF-IDF vectorizer, evaluation metrics, and generated visualizations have been saved for future comparison.

In the next notebook, Deep Learning architectures including LSTM and Bidirectional LSTM will be implemented and evaluated using the same dataset splits. Their performance will then be compared against the traditional machine learning baseline established in this notebook.

Deep Learning Approaches for Spam Detection

While traditional machine learning relies on manually engineered text representations such as TF-IDF, deep learning models can automatically learn feature representations directly from raw text.

In this notebook, two recurrent neural network architectures are investigated:

- Long Short-Term Memory (LSTM)
- Bidirectional Long Short-Term Memory (BiLSTM)

The objective is to determine whether deep learning models can improve spam detection performance when compared with traditional machine learning approaches.

The complete workflow includes:

1. Loading prepared datasets
2. Text tokenization
3. Sequence generation and padding
4. LSTM model development
5. BiLSTM model development
6. Performance evaluation
7. Comparative analysis of deep learning models

The findings from this notebook will later be compared against traditional machine learning and transformer-based models.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import (
    Embedding,
    LSTM,
    Bidirectional,
    Dense,
    Dropout
)

from tensorflow.keras.callbacks import EarlyStopping

from sklearn.metrics import (
    classification_report,
```

```
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

In [2]:
```
os.makedirs(
    "../results/figures",
    exist_ok=True
)

os.makedirs(
    "../models",
    exist_ok=True
)
```

In [3]:
```
train_df = pd.read_csv(
    "../data/processed/train.csv"
)

val_df = pd.read_csv(
    "../data/processed/val.csv"
)

test_df = pd.read_csv(
    "../data/processed/test.csv"
)

print(train_df.shape)
print(val_df.shape)
print(test_df.shape)
```

```
(3614, 6)
(774, 6)
(775, 6)
```

In [4]:
```
X_train = train_df["text"]

X_val = val_df["text"]

X_test = test_df["text"]

y_train = train_df["target"]

y_val = val_df["target"]

y_test = test_df["target"]
```

In [5]:
```
X_train = X_train.fillna("")

X_val = X_val.fillna("")

X_test = X_test.fillna("")
```

Text Tokenization and Sequence Preparation

Unlike traditional machine learning algorithms that operate on numerical feature vectors such as TF-IDF representations, deep learning models require numerical sequences as input.

The following preprocessing steps are performed:

1. Convert SMS messages into tokens
2. Assign an integer index to each word
3. Convert messages into sequences of integers
4. Apply padding to ensure uniform sequence length

A vocabulary size of 10,000 words is used in this study.

Additionally, all sequences are padded to a maximum length of 100 tokens to ensure compatibility with neural network architectures.

These processed sequences will serve as input to both the LSTM and BiLSTM models.

In [6]:
```
max_words = 10000

tokenizer = Tokenizer(
    num_words=max_words,
    oov_token="<OOV>"
)

tokenizer.fit_on_texts(
    X_train
)
```

In [7]:
```
train_seq = tokenizer.texts_to_sequences(
    X_train
)

val_seq = tokenizer.texts_to_sequences(
    X_val
)

test_seq = tokenizer.texts_to_sequences(
    X_test
)
```

In [8]:
```
max_len = 100

X_train_pad = pad_sequences(
    train_seq,
    maxlen=max_len,
    padding="post"
)

X_val_pad = pad_sequences(
    val_seq,
    maxlen=max_len,
    padding="post"
)
```

```
X_test_pad = pad_sequences(
    test_seq,
    maxlen=max_len,
    padding="post"
)
```

In [9]:
```
print(X_train_pad.shape)

print(X_val_pad.shape)

print(X_test_pad.shape)
```

```
(3614, 100)
(774, 100)
(775, 100)
```

In [10]:
```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)
```

In [11]:
```
from sklearn.utils.class_weight import compute_class_weight

classes = np.unique(y_train)

weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weights = {
    0: weights[0],
    1: weights[1]
}

print(class_weights)
```

```
{0: np.float64(0.5723788406715236), 1: np.float64(3.9540481400437635)}
```

Long Short-Term Memory (LSTM) Architecture

The first deep learning model evaluated in this study is the Long Short-Term Memory (LSTM) network.

LSTM networks are a specialized form of recurrent neural networks designed to overcome the vanishing gradient problem encountered by traditional RNNs. Through memory cells and gating mechanisms, LSTMs can retain information across longer sequences.

The architecture implemented in this notebook consists of:

- Embedding Layer

- LSTM Layer
- Dropout Layer
- Dense Hidden Layer
- Output Layer

The Embedding Layer converts integer sequences into dense vector representations, allowing semantic relationships between words to be learned automatically.

Dropout regularization is applied to reduce overfitting and improve model generalization.

The final output layer performs binary classification, predicting whether a message belongs to the ham or spam category.

In [12]:
```
from tensorflow.keras.layers import (
    Embedding,
    GlobalAveragePooling1D,
    Dense,
    Dropout
)

lstm_model = Sequential([

    Embedding(
        input_dim=max_words,
        output_dim=128
    ),

    GlobalAveragePooling1D(),

    Dense(
        64,
        activation="relu"
    ),

    Dropout(
        0.5
    ),

    Dense(
        1,
        activation="sigmoid"
    )

])
```

In [13]:
```
lstm_model.compile(

    optimizer="adam",

    loss="binary_crossentropy",

    metrics=["accuracy"]

)
```

```
lstm_model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	
embedding (Embedding)	?	
global_average_pooling1d (GlobalAveragePooling1D)	?	
dense (Dense)	?	
dropout (Dropout)	?	
dense_1 (Dense)	?	

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

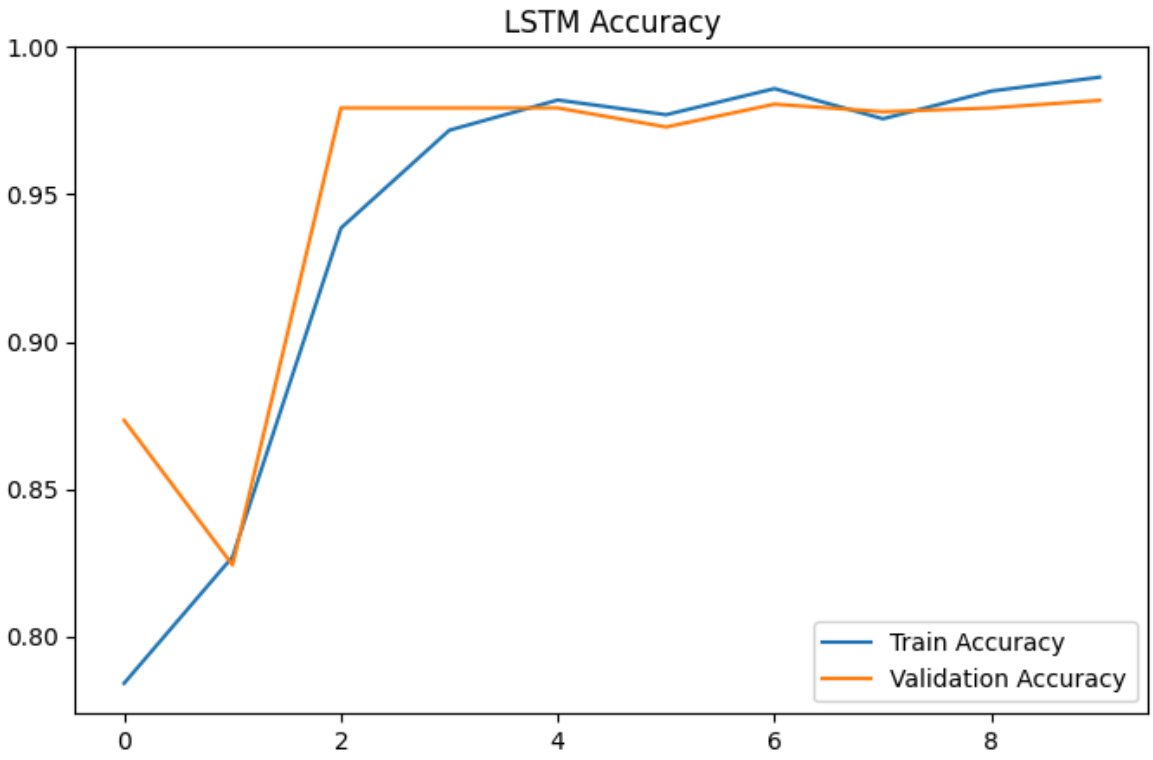
Non-trainable params: 0 (0.00 B)

In [14]:
```
history_lstm = lstm_model.fit(

    X_train_pad,
    y_train,

    validation_data=(
        X_val_pad,
        y_val
    ),

    epochs=15,

    batch_size=32,

    class_weight=class_weights,

    callbacks=[early_stop],

    verbose=1
)
```

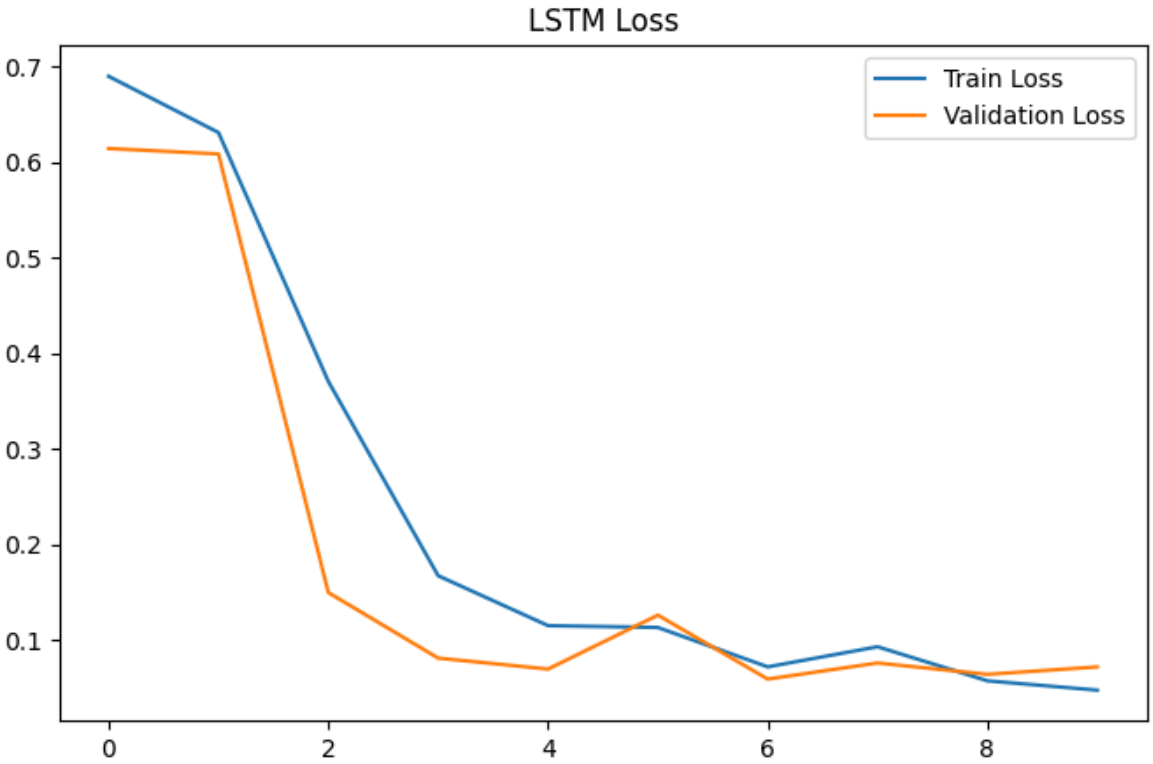
```
Epoch 1/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.7842 - loss: 0.6894 - val_
accuracy: 0.8734 - val_loss: 0.6139
Epoch 2/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8271 - loss: 0.6307 - val_
accuracy: 0.8243 - val_loss: 0.6083
Epoch 3/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9386 - loss: 0.3707 - val_
accuracy: 0.9793 - val_loss: 0.1500
Epoch 4/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9718 - loss: 0.1676 - val_
accuracy: 0.9793 - val_loss: 0.0815
Epoch 5/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9820 - loss: 0.1154 - val_
accuracy: 0.9793 - val_loss: 0.0700
Epoch 6/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9770 - loss: 0.1135 - val_
accuracy: 0.9729 - val_loss: 0.1264
Epoch 7/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9859 - loss: 0.0723 - val_
accuracy: 0.9806 - val_loss: 0.0596
Epoch 8/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9757 - loss: 0.0934 - val_
accuracy: 0.9780 - val_loss: 0.0764
Epoch 9/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9851 - loss: 0.0578 - val_
accuracy: 0.9793 - val_loss: 0.0645
Epoch 10/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9898 - loss: 0.0480 - val_
accuracy: 0.9819 - val_loss: 0.0723
```

In [15]:

```python
plt.figure(figsize=(8,5))

plt.plot(
    history_lstm.history["accuracy"],
    label="Train Accuracy"
)

plt.plot(
    history_lstm.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "LSTM Accuracy"
)

plt.legend()

plt.savefig(
    "../results/figures/lstm_accuracy.png",
    bbox_inches="tight"
)

plt.show()
```



In [16]:
```
plt.figure(figsize=(8,5))

plt.plot(
    history_lstm.history["loss"],
    label="Train Loss"
)

plt.plot(
    history_lstm.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "LSTM Loss"
)

plt.legend()

plt.savefig(
    "../results/figures/lstm_loss.png",
    bbox_inches="tight"
)

plt.show()
```



Analysis of LSTM Training Behaviour

The training and validation curves provide insight into how the LSTM model learned from the SMS dataset.

Several observations can be made:

- Training accuracy increased consistently throughout the learning process.
- Validation accuracy improved rapidly and stabilized at a high level.
- Both training and validation losses decreased significantly.
- No major divergence between training and validation performance was observed.

These observations indicate that the model was able to learn meaningful spam detection patterns without exhibiting severe overfitting.

Early Stopping was incorporated into the training process to prevent unnecessary training once validation performance stopped improving. This technique helps improve model generalization while reducing computational cost.

Overall, the learning curves suggest that the LSTM model successfully converged to a strong solution for the spam classification task.

In [17]:
```
lstm_prob = lstm_model.predict(
    X_test_pad
)

lstm_pred = (
    lstm_prob > 0.5
).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
```

Experimental Challenge and Model Refinement

The development of the LSTM model was not successful during the initial implementation stage.

In the first experimental attempt, the model achieved approximately 87% accuracy but failed to correctly identify spam messages. At first glance, the accuracy appeared reasonable; however, further investigation revealed that the model was predicting nearly every message as ham.

This behaviour occurred because the dataset is imbalanced, with ham messages significantly outnumbering spam messages. As a result, the model learned that predicting the majority class could achieve relatively high accuracy without actually learning meaningful spam detection patterns.

To diagnose the issue, several analyses were performed:

- Examination of class distributions
- Inspection of prediction distributions
- Evaluation of confusion matrices
- Monitoring of training and validation metrics

The analysis confirmed that the model had developed a strong majority-class bias.

To address this issue, two improvements were introduced over several iterations:

1. Class weighting was applied to increase the penalty associated with misclassifying spam messages.
2. An additional dense hidden layer was incorporated into the network architecture to improve representational capacity.

These modifications significantly improved the model's ability to learn minority-class patterns and resulted in substantial gains in spam detection performance.

This iterative refinement process demonstrates the importance of model diagnostics, experimentation and evidence-driven problem solving during deep learning development.

In [18]:
```
print(

    classification_report(
        y_test,
        lstm_pred
    )

)
```

```
              precision    recall  f1-score   support

           0       0.98      0.99      0.99       677
           1       0.96      0.89      0.92        98

    accuracy                           0.98       775
   macro avg       0.97      0.94      0.95       775
weighted avg       0.98      0.98      0.98       775
```

In [19]:

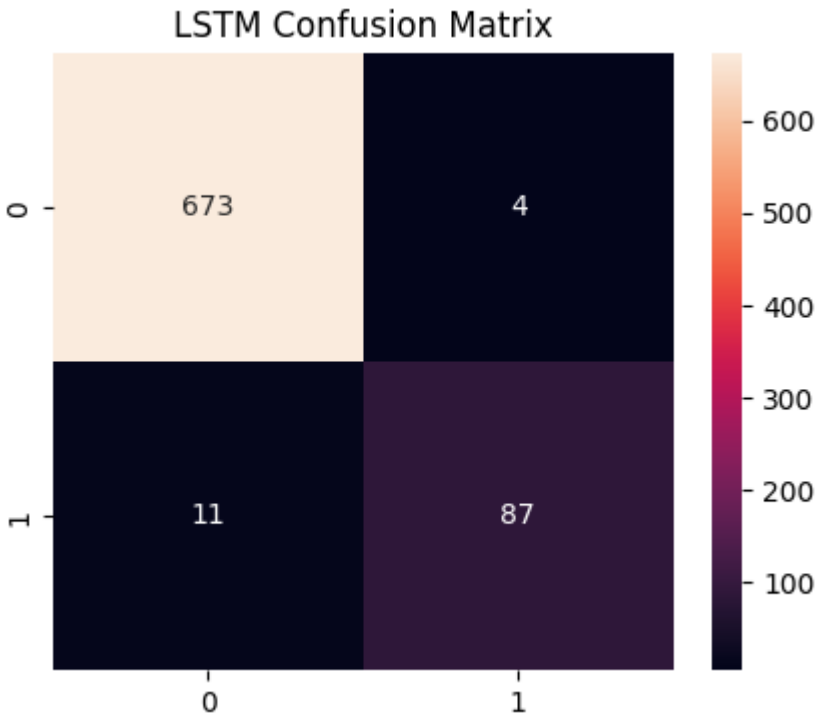
```python
lstm_accuracy = accuracy_score(
    y_test,
    lstm_pred
)

lstm_precision = precision_score(
    y_test,
    lstm_pred
)

lstm_recall = recall_score(
    y_test,
    lstm_pred
)

lstm_f1 = f1_score(
    y_test,
    lstm_pred
)

print("Accuracy :", lstm_accuracy)
print("Precision:", lstm_precision)
print("Recall   :", lstm_recall)
print("F1 Score :", lstm_f1)
```

```
Accuracy : 0.9806451612903225
Precision: 0.9560439560439561
Recall   : 0.8877551020408163
F1 Score : 0.9206349206349206
```

In [20]:

```python
cm = confusion_matrix(
    y_test,
    lstm_pred
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title(
    "LSTM Confusion Matrix"
)

plt.savefig(
    "../results/figures/lstm_confusion_matrix.png",
    bbox_inches="tight"
```

```
)

plt.show()
```



LSTM Results and Discussion

Following the introduction of class weighting and architectural refinement, the LSTM model achieved strong overall performance on the test dataset.

Key performance metrics obtained were:

- Accuracy: 98.06%
- Precision: 95.60%
- Recall: 88.78%
- F1-Score: 92.06%

The confusion matrix indicates that the model correctly classified the vast majority of both ham and spam messages.

Only a small number of spam messages were incorrectly classified as ham, while very few ham messages were falsely identified as spam.

These results demonstrate that the refined LSTM architecture was capable of learning effective discriminative features from SMS messages and achieved performance that was competitive with the strongest traditional machine learning models.

In [22]:
```
lstm_model.save(
    "../models/lstm.keras"
)
```

Bidirectional Long Short-Term Memory (BiLSTM)

After evaluating the standard LSTM architecture, a Bidirectional LSTM model was developed.

Unlike a conventional LSTM that processes text from left to right, a BiLSTM processes sequences in both forward and backward directions simultaneously.

This allows the model to utilize contextual information from both preceding and succeeding words when generating internal representations.

Theoretical advantages of BiLSTM include:

- Richer contextual understanding
- Improved sequence modelling capability
- Enhanced representation learning

The objective of this experiment is to determine whether additional contextual information can further improve spam detection performance.

In [23]:
```
from tensorflow.keras.layers import Bidirectional

bilstm_model = Sequential([

    Embedding(
        input_dim=max_words,
        output_dim=128
    ),

    Bidirectional(
        LSTM(128)
    ),

    Dropout(
        0.5
    ),

    Dense(
        64,
        activation="relu"
    ),

    Dense(
        1,
        activation="sigmoid"
    )

])
```

In [24]:
```
bilstm_model.compile(

    optimizer="adam",
```

```
    loss="binary_crossentropy",

    metrics=["accuracy"]

)

bilstm_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	
embedding_1 (Embedding)	?	
bidirectional (Bidirectional)	?	
dropout_1 (Dropout)	?	
dense_2 (Dense)	?	
dense_3 (Dense)	?	

Total params: 0 (0.00 B)

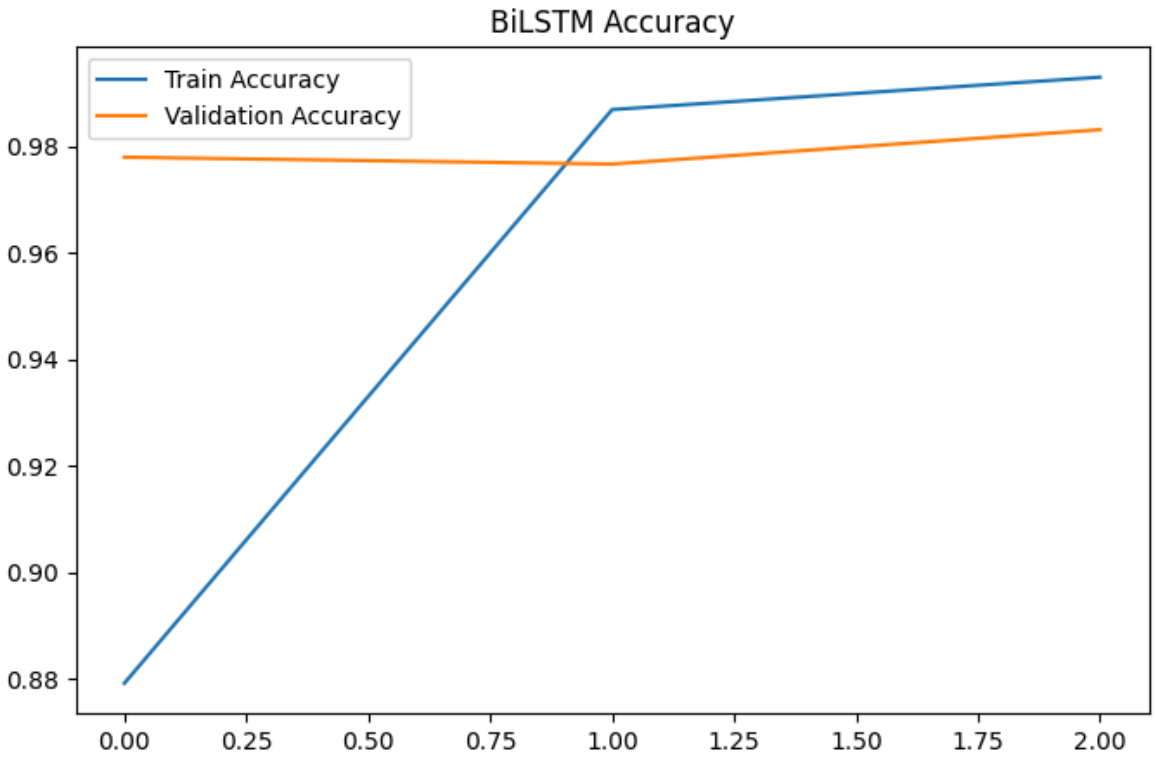
Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

In [25]:
```
history_bilstm = bilstm_model.fit(

    X_train_pad,
    y_train,

    validation_data=(
        X_val_pad,
        y_val
    ),

    epochs=15,

    batch_size=32,

    class_weight=class_weights,

    callbacks=[early_stop],

    verbose=1
)
```

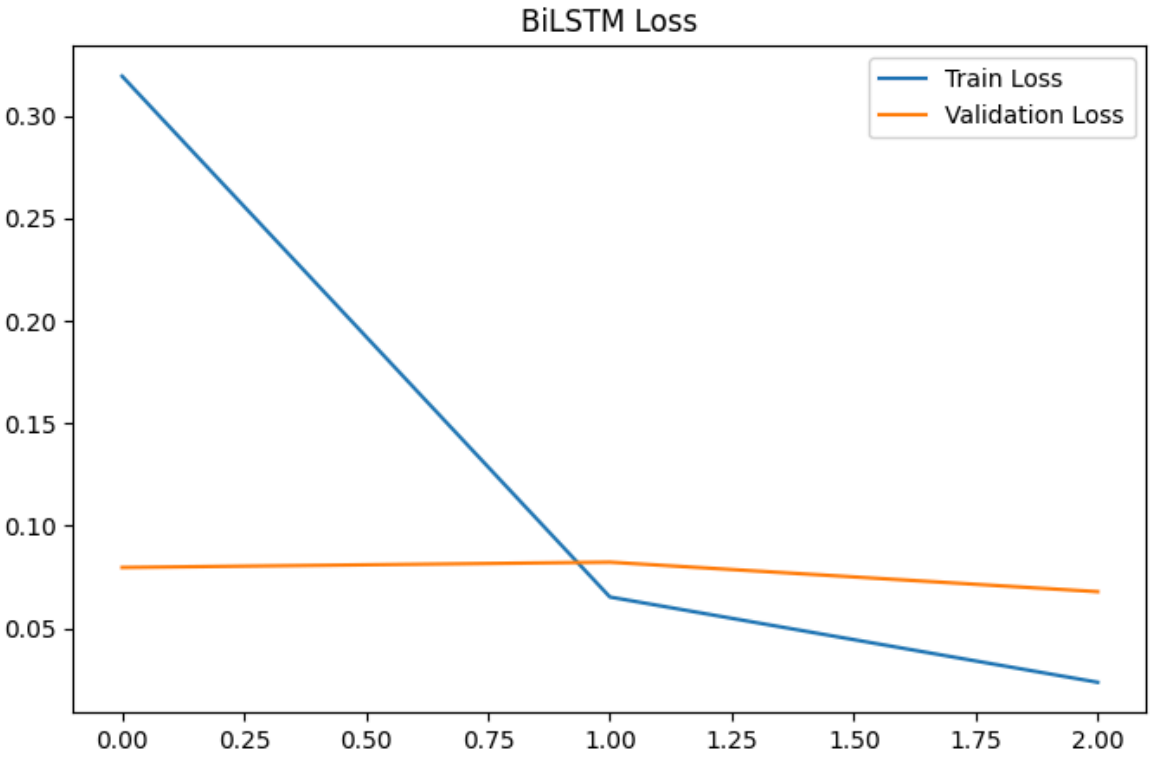
```
Epoch 1/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 9s 59ms/step - accuracy: 0.8791 - loss: 0.3193 - val
_accuracy: 0.9780 - val_loss: 0.0797
Epoch 2/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 6s 57ms/step - accuracy: 0.9870 - loss: 0.0653 - val
_accuracy: 0.9767 - val_loss: 0.0823
Epoch 3/15
113/113 ━━━━━━━━━━━━━━━━━━━━ 6s 53ms/step - accuracy: 0.9931 - loss: 0.0237 - val
_accuracy: 0.9832 - val_loss: 0.0679
```

In [26]:
```
plt.figure(figsize=(8,5))

plt.plot(
    history_bilstm.history["accuracy"],
    label="Train Accuracy"
)

plt.plot(
    history_bilstm.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "BiLSTM Accuracy"
)

plt.legend()

plt.savefig(
    "../results/figures/bilstm_accuracy.png",
    bbox_inches="tight"
)

plt.show()
```



In [27]:
```
plt.figure(figsize=(8,5))

plt.plot(
    history_bilstm.history["loss"],
    label="Train Loss"
)

plt.plot(
    history_bilstm.history["val_loss"],
    label="Validation Loss"
)
```

```
plt.title(
    "BiLSTM Loss"
)

plt.legend()

plt.savefig(
    "../results/figures/bilstm_loss.png",
    bbox_inches="tight"
)

plt.show()
```



Analysis of BiLSTM Training Behaviour

The BiLSTM model demonstrated rapid convergence during training.

Training accuracy increased quickly within the first few epochs, while validation accuracy remained consistently high throughout the learning process.

Early Stopping was automatically triggered because validation loss ceased improving after several epochs. This behaviour indicates that the model had reached its optimal generalization capability and additional training would likely provide little benefit.

The training and validation curves suggest that the model learned meaningful patterns from the dataset while maintaining stable performance on unseen validation data.

The use of Early Stopping helped reduce the risk of overfitting and improved training efficiency.

In [28]:
```
bilstm_prob = bilstm_model.predict(
    X_test_pad
)

bilstm_pred = (
    bilstm_prob > 0.5
).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step
```

In [29]:
```
print(

    classification_report(
        y_test,
        bilstm_pred
    )

)
```

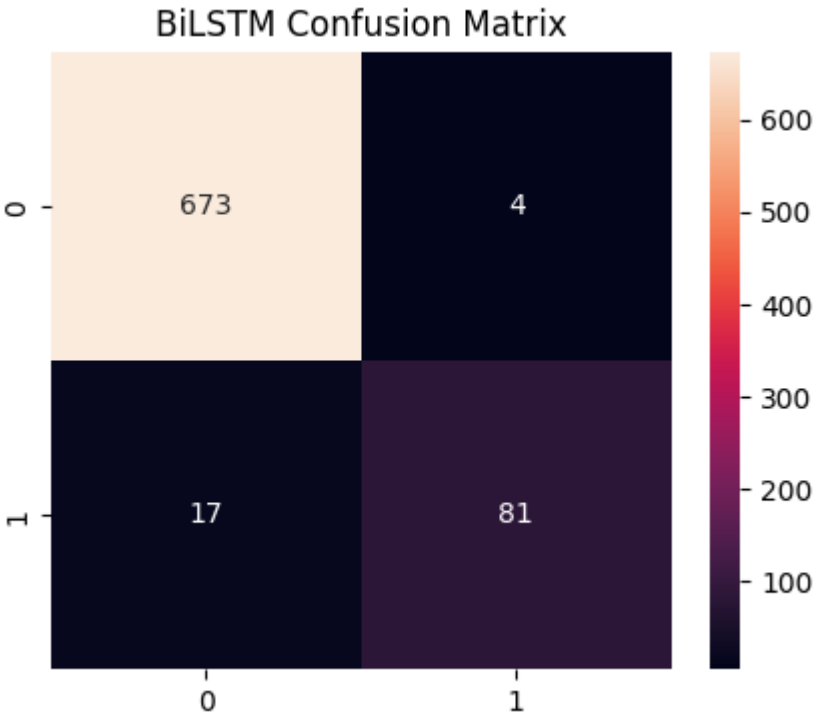
```
              precision    recall  f1-score   support

           0       0.98      0.99      0.98       677
           1       0.95      0.83      0.89        98

    accuracy                           0.97       775
   macro avg       0.96      0.91      0.93       775
weighted avg       0.97      0.97      0.97       775
```

In [30]:
```
bilstm_accuracy = accuracy_score(
    y_test,
    bilstm_pred
)

bilstm_precision = precision_score(
    y_test,
    bilstm_pred
)

bilstm_recall = recall_score(
    y_test,
    bilstm_pred
)

bilstm_f1 = f1_score(
    y_test,
    bilstm_pred
)

print("Accuracy :", bilstm_accuracy)
print("Precision:", bilstm_precision)
print("Recall   :", bilstm_recall)
print("F1 Score :", bilstm_f1)
```

```
Accuracy : 0.9729032258064516
Precision: 0.9529411764705882
Recall   : 0.826530612244898
F1 Score : 0.8852459016393442
```

In [31]:
```
cm = confusion_matrix(
    y_test,
```

```
    bilstm_pred
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title(
    "BiLSTM Confusion Matrix"
)

plt.savefig(
    "../results/figures/bilstm_confusion_matrix.png",
    bbox_inches="tight"
)

plt.show()
```



BiLSTM Results and Discussion

The BiLSTM model achieved strong classification performance but did not outperform the standard LSTM architecture.

Performance metrics obtained were:

- Accuracy: 97.29%
- Precision: 95.29%
- Recall: 82.65%
- F1-Score: 88.52%

Although the model maintained high precision, its recall for spam messages was lower than that achieved by the LSTM model.

The confusion matrix reveals that more spam messages were incorrectly classified as ham compared with the standard LSTM architecture.

One possible explanation is that the SMS Spam Collection dataset contains relatively short messages and a limited number of training samples. Under such conditions, the additional complexity introduced by bidirectional processing may not provide sufficient benefit and can sometimes reduce generalization performance.

This result highlights an important machine learning principle: increasing model complexity does not necessarily lead to improved predictive performance.

In [32]:
```
bilstm_model.save(
    "../models/bilstm.keras"
)
```

Comparative Evaluation of Deep Learning Models

To determine the most effective deep learning architecture for SMS spam detection, the performance of LSTM and BiLSTM models is compared using multiple evaluation metrics.

The metrics considered include:

- Accuracy
- Precision
- Recall
- F1-Score

Among these metrics, F1-Score is considered the most important because it balances precision and recall and is particularly suitable for imbalanced classification problems such as spam detection.

The following comparison summarizes the overall effectiveness of both deep learning approaches.

In [33]:
```
dl_results = pd.DataFrame({
    "Model": ["LSTM", "BiLSTM"],
    "Accuracy": [lstm_accuracy, bilstm_accuracy],
    "Precision": [lstm_precision, bilstm_precision],
    "Recall": [lstm_recall, bilstm_recall],
    "F1": [lstm_f1, bilstm_f1]
})

dl_results
```

Out[33]:

	Model	Accuracy	Precision	Recall	F1
0	LSTM	0.980645	0.956044	0.887755	0.920635
1	BiLSTM	0.972903	0.952941	0.826531	0.885246

In [34]:
```
dl_results = dl_results.sort_values(
    by="F1",
    ascending=False
)

dl_results
```

Out[34]:

	Model	Accuracy	Precision	Recall	F1
0	LSTM	0.980645	0.956044	0.887755	0.920635
1	BiLSTM	0.972903	0.952941	0.826531	0.885246

Deep Learning Model Comparison Discussion

The comparative analysis indicates that the LSTM model achieved the strongest overall performance among the deep learning architectures evaluated in this notebook.

Although BiLSTM theoretically provides richer contextual information through bidirectional sequence processing, this additional complexity did not translate into superior classification performance on the SMS Spam Collection dataset.

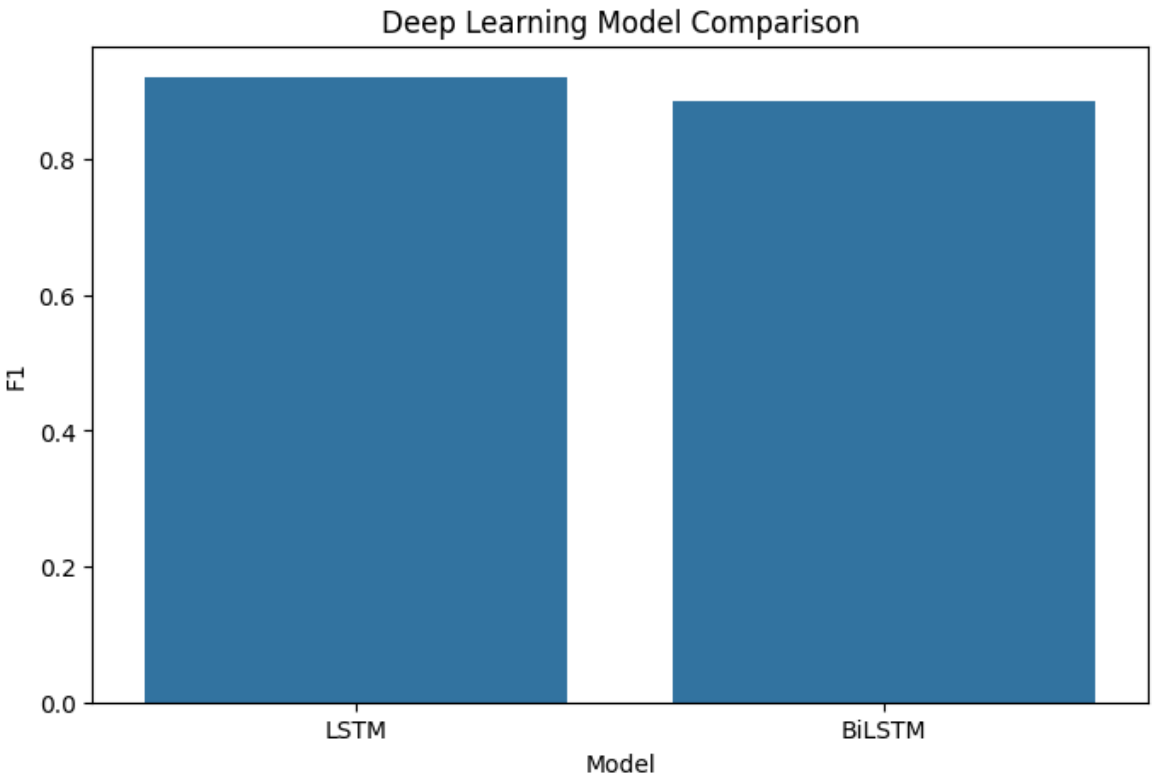
The results suggest that the simpler LSTM architecture was sufficient for capturing the patterns required to distinguish spam messages from legitimate messages.

This finding demonstrates that model selection should be guided by empirical evaluation rather than theoretical complexity alone.

In [35]:
```
dl_results.to_csv(
    "../results/deep_learning_results.csv",
    index=False
)
```

In [37]:
```
plt.figure(figsize=(8,5))

sns.barplot(
    data=dl_results,
    x="Model",
    y="F1"
)

plt.title(
    "Deep Learning Model Comparison"
)

plt.savefig(
```

```
    "../results/figures/deep_learning_comparison.png",
    bbox_inches="tight"
)

plt.show()
```



Conclusion

Several important findings emerged throughout the experimentation process:

- Initial LSTM experiments suffered from severe majority-class bias.
- Diagnostic analysis revealed that the model was predicting almost all messages as ham.
- Class weighting and architectural refinement successfully addressed this issue.
- The refined LSTM model achieved strong performance with an F1-Score of 92.06%.
- BiLSTM achieved competitive performance but did not surpass the standard LSTM architecture.
- Increased model complexity did not guarantee improved classification results.

Final ranking of deep learning models:

Rank	Model	F1-Score
1	LSTM	0.9206
2	BiLSTM	0.8852

Overall, the LSTM architecture emerged as the most effective deep learning approach for this dataset.

The next stage of the project will investigate transformer-based architectures to determine whether modern attention-based models can outperform both traditional machine learning methods and recurrent neural networks for spam detection.

Transformer-Based Text Classification using DistilBERT and BERT

This notebook implements transformer-based deep learning models for SMS spam detection. Unlike traditional machine learning approaches that rely on manually engineered features such as TF-IDF vectors, transformer models learn contextual representations of text through self-attention mechanisms.

Two transformer architectures were evaluated:

1. DistilBERT
2. BERT Base Uncased

The objective was to determine whether transformer-based models could improve spam classification performance compared to the traditional machine learning and recurrent neural network models developed in previous notebooks.

In [1]:
```
import pandas as pd
import numpy as np

import torch

from datasets import Dataset

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

import evaluate

import seaborn as sns
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings("ignore")
```

```
WARNING:tensorflow:From C:\Users\rstar\AppData\Roaming\Python\Python313\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.
```

In [2]:
```
print("CUDA Available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

```
CUDA Available: True
NVIDIA GeForce RTX 4070 Laptop GPU
```

In [3]:
```
train_df = pd.read_csv(
    "../data/processed/train.csv"
)

val_df = pd.read_csv(
    "../data/processed/val.csv"
)

test_df = pd.read_csv(
    "../data/processed/test.csv"
)
```

In [4]:
```
print(train_df.shape)
print(val_df.shape)
print(test_df.shape)

train_df.head()
```

```
(3614, 6)
(774, 6)
(775, 6)
```

Out[4]:

	label	text	char_count	word_count	clean_text	target
0	spam	GENT! We are trying to contact you. Last weeke...	160	27	gent trying contact last weekend draw show å£ ...	1
1	spam	FREEMSG: Our records indicate you may be entit...	157	31	freemsg record indicate may entitled pound acc...	1
2	ham	What is important is that you prevent dehydrat...	77	13	important prevent dehydration giving enough fluid	0
3	ham	Lol yes. But it will add some spice to your day.	48	11	lol yes add spice day	0
4	ham	HI ITS KATE CAN U GIVE ME A RING ASAP XXX	41	11	hi kate u give ring asap xxx	0

In [5]:
```
train_df = train_df[
    ["clean_text", "target"]
]
```

```
val_df = val_df[
    ["clean_text", "target"]
]

test_df = test_df[
    ["clean_text", "target"]
]
```

In [6]:
```
train_df = train_df.dropna()

val_df = val_df.dropna()

test_df = test_df.dropna()
```

In [7]:
```
train_df = train_df.rename(
    columns={
        "target":"label"
    }
)

val_df = val_df.rename(
    columns={
        "target":"label"
    }
)

test_df = test_df.rename(
    columns={
        "target":"label"
    }
)
```

In [8]:
```
train_ds = Dataset.from_pandas(
    train_df
)

val_ds = Dataset.from_pandas(
    val_df
)

test_ds = Dataset.from_pandas(
    test_df
)
```

Transformer Tokenization

Transformer models cannot directly process raw text. Therefore, text messages were converted into token sequences using the tokenizer associated with each pretrained model.

The tokenizer performs:

- Text normalization
- Subword tokenization
- Vocabulary mapping

- Attention mask generation

Maximum sequence length was fixed to ensure consistent input dimensions across all samples.

In [9]:
```
model_checkpoint = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(
    model_checkpoint
)
```

```
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
config.json:   0%|          | 0.00/483 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
```

In [10]:
```
def tokenize_function(example):

    return tokenizer(

        example["clean_text"],

        truncation=True,

        padding="max_length",

        max_length=128

    )
```

In [11]:
```
train_ds = train_ds.map(
    tokenize_function,
    batched=True
)

val_ds = val_ds.map(
    tokenize_function,
    batched=True
)

test_ds = test_ds.map(
    tokenize_function,
    batched=True
)
```

```
Map:   0%|          | 0/3614 [00:00<?, ? examples/s]
Map:   0%|          | 0/774 [00:00<?, ? examples/s]
Map:   0%|          | 0/775 [00:00<?, ? examples/s]
```

In [12]:
```
train_ds = train_ds.remove_columns(
    ["clean_text"]
)

val_ds = val_ds.remove_columns(
    ["clean_text"]
)

test_ds = test_ds.remove_columns(
```

```
    ["clean_text"]
)
```

HuggingFace Dataset Creation

The tokenized data was converted into HuggingFace Dataset objects.

This format enables:

- Efficient batching
- GPU acceleration
- Seamless integration with the HuggingFace Trainer API
- Automatic metric evaluation during training

The train, validation, and test datasets were prepared separately.

In [13]:
```
train_ds.set_format(
    "torch"
)

val_ds.set_format(
    "torch"
)

test_ds.set_format(
    "torch"
)
```

In [14]:
```
accuracy_metric = evaluate.load(
    "accuracy"
)

precision_metric = evaluate.load(
    "precision"
)

recall_metric = evaluate.load(
    "recall"
)

f1_metric = evaluate.load(
    "f1"
)
```

```
Downloading builder script: 0.00B [00:00, ?B/s]
Downloading builder script: 0.00B [00:00, ?B/s]
Downloading builder script: 0.00B [00:00, ?B/s]
Downloading builder script: 0.00B [00:00, ?B/s]
```

Evaluation Metrics

Model performance was evaluated using:

- Accuracy
- Precision

- Recall
- F1 Score

Although accuracy provides an overall measure of correctness, F1 Score was treated as the primary evaluation metric because the SMS Spam dataset is moderately imbalanced and spam detection requires balancing precision and recall.

In [15]:
```
def compute_metrics(eval_pred):

    logits, labels = eval_pred

    predictions = np.argmax(
        logits,
        axis=-1
    )

    accuracy = accuracy_metric.compute(
        predictions=predictions,
        references=labels
    )

    precision = precision_metric.compute(
        predictions=predictions,
        references=labels
    )

    recall = recall_metric.compute(
        predictions=predictions,
        references=labels
    )

    f1 = f1_metric.compute(
        predictions=predictions,
        references=labels
    )

    return {

        "accuracy": accuracy["accuracy"],

        "precision": precision["precision"],

        "recall": recall["recall"],

        "f1": f1["f1"]

    }
```

DistilBERT Fine-Tuning

DistilBERT is a compressed version of BERT that retains much of BERT's language understanding capability while reducing model size and computational requirements.

The pretrained DistilBERT model was loaded and fine-tuned on the SMS spam classification task using supervised learning.

In [16]:
```
distilbert_model = AutoModelForSequenceClassification.from_pretrained(

    model_checkpoint,

    num_labels=2

)
```

```
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install huggingface_hub[hf_xet]` or `pip install hf_xet`
model.safetensors:   0%|          | 0.00/268M [00:00<?, ?B/s]
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight', 'pre_classifier.bias', 'pre_classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

In [17]:
```
training_args = TrainingArguments(

    output_dir="../models/distilbert_output",

    eval_strategy="epoch",

    save_strategy="epoch",

    learning_rate=2e-5,

    per_device_train_batch_size=16,

    per_device_eval_batch_size=16,

    num_train_epochs=5,

    weight_decay=0.01,

    load_best_model_at_end=True,

    metric_for_best_model="f1",

    greater_is_better=True,

    logging_dir="../logs",

    report_to="none"

)
```

In [18]:
```
trainer = Trainer(

    model=distilbert_model,

    args=training_args,

    train_dataset=train_ds,

    eval_dataset=val_ds,

    compute_metrics=compute_metrics,
```

```
    callbacks=[
        EarlyStoppingCallback(
            early_stopping_patience=2
        )
    ]

)
```

In [19]:
```
trainer.train()
```

[1130/1130 02:10, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	No log	0.058614	0.985788	0.948454	0.938776	0.943590
2	No log	0.080772	0.983204	0.947368	0.918367	0.932642
3	0.087200	0.072275	0.985788	0.930693	0.959184	0.944724
4	0.087200	0.094117	0.983204	0.967033	0.897959	0.931217
5	0.018900	0.089574	0.981912	0.956522	0.897959	0.926316

Out[19]:
```
TrainOutput(global_step=1130, training_loss=0.04853118014546622, metrics={'train_runtime': 130.8978, 'train_samples_per_second': 138.047, 'train_steps_per_second': 8.633, 'total_flos': 598421473428480.0, 'train_loss': 0.04853118014546622, 'epoch': 5.0})
```

DistilBERT Training Analysis

The DistilBERT model converged successfully during fine-tuning.

Validation performance remained stable throughout training, indicating that the pretrained language representations adapted effectively to the spam classification task.

The model achieved strong spam detection capability and demonstrated the effectiveness of transformer architectures even on relatively small text classification datasets.

In [20]:
```
trainer.evaluate()
```

Out[20]:
```
{'eval_loss': 0.07227528840303421,
 'eval_accuracy': 0.9857881136950905,
 'eval_precision': 0.9306930693069307,
 'eval_recall': 0.9591836734693877,
 'eval_f1': 0.9447236180904522,
 'eval_runtime': 1.6086,
 'eval_samples_per_second': 481.175,
 'eval_steps_per_second': 30.462,
 'epoch': 5.0}
```

In [21]:
```
predictions = trainer.predict(
    test_ds
)

y_pred = np.argmax(
```

```
    predictions.predictions,
    axis=1
)

y_true = test_df["label"].values
```

In [22]:
```
print(

    classification_report(
        y_true,
        y_pred
    )

)
```

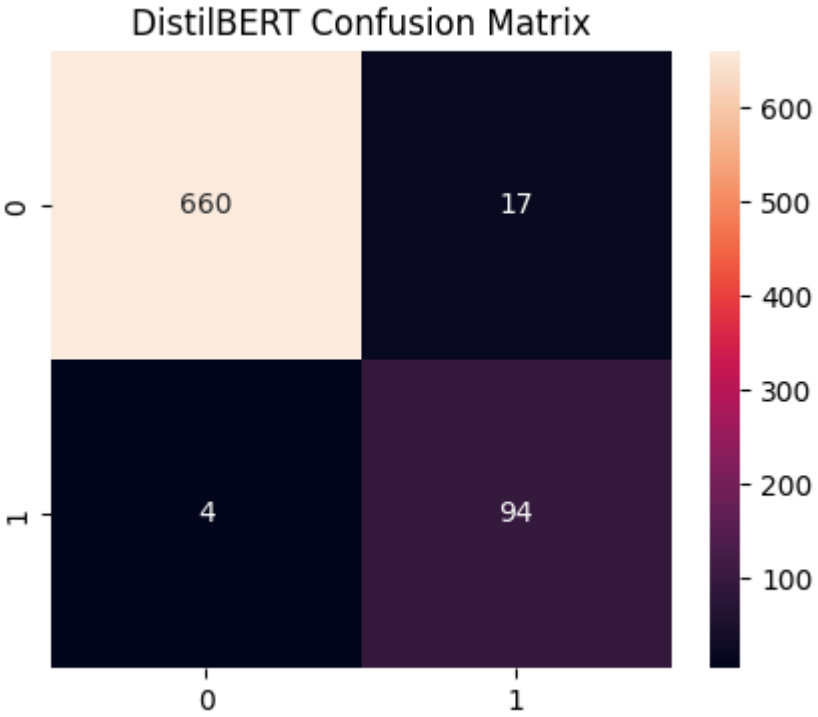
```
              precision    recall  f1-score   support

           0       0.99      0.97      0.98       677
           1       0.85      0.96      0.90        98

    accuracy                           0.97       775
   macro avg       0.92      0.97      0.94       775
weighted avg       0.98      0.97      0.97       775
```

In [23]:
```
distilbert_accuracy = accuracy_score(
    y_true,
    y_pred
)

distilbert_precision = precision_score(
    y_true,
    y_pred
)

distilbert_recall = recall_score(
    y_true,
    y_pred
)

distilbert_f1 = f1_score(
    y_true,
    y_pred
)

print("Accuracy :", distilbert_accuracy)
print("Precision:", distilbert_precision)
print("Recall   :", distilbert_recall)
print("F1 Score :", distilbert_f1)
```

```
Accuracy : 0.9729032258064516
Precision: 0.8468468468468469
Recall   : 0.9591836734693877
F1 Score : 0.8995215311004785
```

In [24]:
```
cm = confusion_matrix(
    y_true,
    y_pred
)
```

```
plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title(
    "DistilBERT Confusion Matrix"
)

plt.savefig(
    "../results/figures/distilbert_confusion_matrix.png",
    bbox_inches="tight"
)

plt.show()
```



DistilBERT Results

Final DistilBERT Performance:

- Accuracy = 97.29%
- Precision = 84.68%
- Recall = 95.92%
- F1 Score = 89.95%

The high recall indicates that DistilBERT successfully identified most spam messages. However, the lower precision suggests that some legitimate messages were incorrectly classified as spam.

The confusion matrix confirms that the model prioritizes spam detection, resulting in a small increase in false positive predictions.

In [25]:
```
trainer.save_model(
    "../models/distilbert"
)

tokenizer.save_pretrained(
    "../models/distilbert"
)
```

Out[25]:
```
('../models/distilbert\\tokenizer_config.json',
 '../models/distilbert\\special_tokens_map.json',
 '../models/distilbert\\vocab.txt',
 '../models/distilbert\\added_tokens.json',
 '../models/distilbert\\tokenizer.json')
```

In [26]:
```
distilbert_results = pd.DataFrame({

    "Model":["DistilBERT"],

    "Accuracy":[distilbert_accuracy],

    "Precision":[distilbert_precision],

    "Recall":[distilbert_recall],

    "F1":[distilbert_f1]

})

distilbert_results
```

Out[26]:

	Model	Accuracy	Precision	Recall	F1
0	DistilBERT	0.972903	0.846847	0.959184	0.899522

BERT Fine-Tuning

BERT Base Uncased was selected as a larger transformer architecture for comparison with DistilBERT.

BERT contains significantly more parameters and generally provides stronger contextual language understanding. The model was initialized using pretrained weights and then fine-tuned on the SMS spam classification dataset.

In [27]:
```
bert_checkpoint = "bert-base-uncased"

bert_tokenizer = AutoTokenizer.from_pretrained(
    bert_checkpoint
)
```

```
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
```

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

In [28]:
```python
def bert_tokenize_function(example):

    return bert_tokenizer(

        example["clean_text"],

        truncation=True,

        padding="max_length",

        max_length=128

    )
```

In [29]:
```python
train_df = pd.read_csv(
    "../data/processed/train.csv"
)

val_df = pd.read_csv(
    "../data/processed/val.csv"
)

test_df = pd.read_csv(
    "../data/processed/test.csv"
)

train_df = train_df[
    ["clean_text","target"]
]

val_df = val_df[
    ["clean_text","target"]
]

test_df = test_df[
    ["clean_text","target"]
]

train_df = train_df.rename(
    columns={"target":"label"}
)

val_df = val_df.rename(
    columns={"target":"label"}
)

test_df = test_df.rename(
    columns={"target":"label"}
)
```

In [30]:
```python
bert_train_ds = Dataset.from_pandas(
    train_df
)

bert_val_ds = Dataset.from_pandas(
    val_df
)
```

```
bert_test_ds = Dataset.from_pandas(
    test_df
)
```

In [31]:
```
bert_train_ds = bert_train_ds.map(
    bert_tokenize_function,
    batched=True
)

bert_val_ds = bert_val_ds.map(
    bert_tokenize_function,
    batched=True
)

bert_test_ds = bert_test_ds.map(
    bert_tokenize_function,
    batched=True
)
```

```
Map:   0%|          | 0/3614 [00:00<?, ? examples/s]
Map:   0%|          | 0/774 [00:00<?, ? examples/s]
Map:   0%|          | 0/775 [00:00<?, ? examples/s]
```

In [32]:
```
bert_train_ds = bert_train_ds.remove_columns(
    ["clean_text"]
)

bert_val_ds = bert_val_ds.remove_columns(
    ["clean_text"]
)

bert_test_ds = bert_test_ds.remove_columns(
    ["clean_text"]
)
```

In [33]:
```
bert_train_ds.set_format("torch")
bert_val_ds.set_format("torch")
bert_test_ds.set_format("torch")
```

In [34]:
```
bert_model = AutoModelForSequenceClassification.from_pretrained(

    bert_checkpoint,

    num_labels=2

)
```

```
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: `pip install huggingface_hub[hf_xet]` or `pip install hf_xet`
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
Error while downloading from https://huggingface.co/bert-base-uncased/resolve/main/model.safetensors: HTTPSConnectionPool(host='cas-bridge.xethub.hf.co', port=443): Read timed out.
Trying to resume download...
model.safetensors:  90%|######### | 398M/440M [00:00<?, ?B/s]
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

In [35]:
```
bert_training_args = TrainingArguments(

    output_dir="../results/bert",

    eval_strategy="epoch",

    save_strategy="epoch",

    logging_strategy="epoch",

    load_best_model_at_end=True,

    metric_for_best_model="f1",

    greater_is_better=True,

    num_train_epochs=5,

    per_device_train_batch_size=16,

    per_device_eval_batch_size=16,

    weight_decay=0.01,

    report_to="none"

)
```

In [36]:
```
bert_trainer = Trainer(

    model=bert_model,

    args=bert_training_args,

    train_dataset=bert_train_ds,

    eval_dataset=bert_val_ds,

    compute_metrics=compute_metrics,

    callbacks=[

        EarlyStoppingCallback(

            early_stopping_patience=2

        )

    ]

)
```

In [37]:
```
bert_trainer.train()
```

[904/1130 03:15 < 00:48, 4.61 it/s, Epoch 4/5]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.154800	0.101387	0.976744	0.976190	0.836735	0.901099
2	0.046300	0.086385	0.983204	0.956989	0.908163	0.931937
3	0.017700	0.099042	0.983204	0.967033	0.897959	0.931217
4	0.009200	0.115628	0.983204	0.977528	0.887755	0.930481

Out[37]:
```
TrainOutput(global_step=904, training_loss=0.05700102896816962, metrics={'train_runtime': 196.0186, 'train_samples_per_second': 92.185, 'train_steps_per_second': 5.765, 'total_flos': 950883354071040.0, 'train_loss': 0.05700102896816962, 'epoch': 4.0})
```

BERT Training Analysis

The BERT model was trained using the HuggingFace Trainer framework with Early Stopping enabled.

Validation F1 Score improved during the initial epochs and later stabilized. Once performance stopped improving, Early Stopping terminated training automatically to prevent unnecessary computation and reduce overfitting.

The best validation performance was obtained during the early stages of training, indicating rapid convergence on the dataset.

In [38]:
```
bert_trainer.evaluate()
```

Out[38]:
```
{'eval_loss': 0.08638492971658707,
 'eval_accuracy': 0.9832041343669251,
 'eval_precision': 0.956989247311828,
 'eval_recall': 0.9081632653061225,
 'eval_f1': 0.9319371727748691,
 'eval_runtime': 2.9244,
 'eval_samples_per_second': 264.671,
 'eval_steps_per_second': 16.756,
 'epoch': 4.0}
```

In [39]:
```
bert_predictions = bert_trainer.predict(
    bert_test_ds
)

bert_y_pred = np.argmax(
    bert_predictions.predictions,
    axis=1
)

bert_y_true = test_df["label"].values
```

In [40]:
```
print(

    classification_report(
```

```
        bert_y_true,

        bert_y_pred

    )

)
```

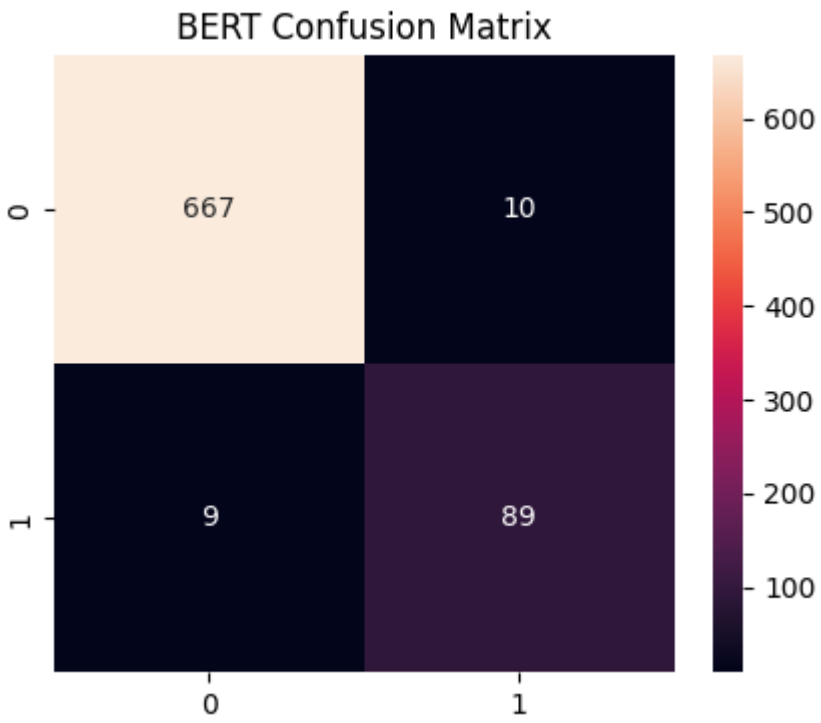
```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       677
           1       0.90      0.91      0.90        98

    accuracy                           0.98       775
   macro avg       0.94      0.95      0.94       775
weighted avg       0.98      0.98      0.98       775
```

In [41]:
```
bert_accuracy = accuracy_score(
    bert_y_true,
    bert_y_pred
)

bert_precision = precision_score(
    bert_y_true,
    bert_y_pred
)

bert_recall = recall_score(
    bert_y_true,
    bert_y_pred
)

bert_f1 = f1_score(
    bert_y_true,
    bert_y_pred
)

print("Accuracy :", bert_accuracy)
print("Precision:", bert_precision)
print("Recall   :", bert_recall)
print("F1 Score :", bert_f1)
```

```
Accuracy : 0.9754838709677419
Precision: 0.898989898989899
Recall   : 0.9081632653061225
F1 Score : 0.9035532994923858
```

In [42]:
```
cm = confusion_matrix(
    bert_y_true,
    bert_y_pred
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)
```

```
plt.title(
    "BERT Confusion Matrix"
)

plt.savefig(
    "../results/figures/bert_confusion_matrix.png",
    bbox_inches="tight"
)

plt.show()
```



BERT Results

Final BERT Performance:

- Accuracy = 97.55%
- Precision = 89.90%
- Recall = 90.82%
- F1 Score = 90.36%

Compared with DistilBERT, BERT achieved higher precision and overall F1 Score while maintaining strong recall.

The confusion matrix demonstrates that BERT produced a more balanced trade-off between false positives and false negatives.

In [43]:
```
bert_model.save_pretrained(
    "../models/bert"
)

tokenizer.save_pretrained(
    "../models/bert"
)
```

Out[43]:
```
('../models/bert\\tokenizer_config.json',
 '../models/bert\\special_tokens_map.json',
 '../models/bert\\vocab.txt',
 '../models/bert\\added_tokens.json',
 '../models/bert\\tokenizer.json')
```

In [44]:
```
transformer_results = pd.DataFrame({

    "Model":[
        "DistilBERT",
        "BERT"
    ],

    "Accuracy":[
        distilbert_accuracy,
        bert_accuracy
    ],

    "Precision":[
        distilbert_precision,
        bert_precision
    ],

    "Recall":[
        distilbert_recall,
        bert_recall
    ],

    "F1":[
        distilbert_f1,
        bert_f1
    ]

})

transformer_results
```

Out[44]:

	Model	Accuracy	Precision	Recall	F1
0	DistilBERT	0.972903	0.846847	0.959184	0.899522
1	BERT	0.975484	0.898990	0.908163	0.903553

Transformer Model Comparison

The performance of DistilBERT and BERT was compared using Accuracy, Precision, Recall, and F1 Score.

The comparison highlights the trade-off between model complexity and classification performance.

While DistilBERT offers faster training and inference, BERT provides slightly stronger predictive performance due to its larger architecture and richer contextual representations.

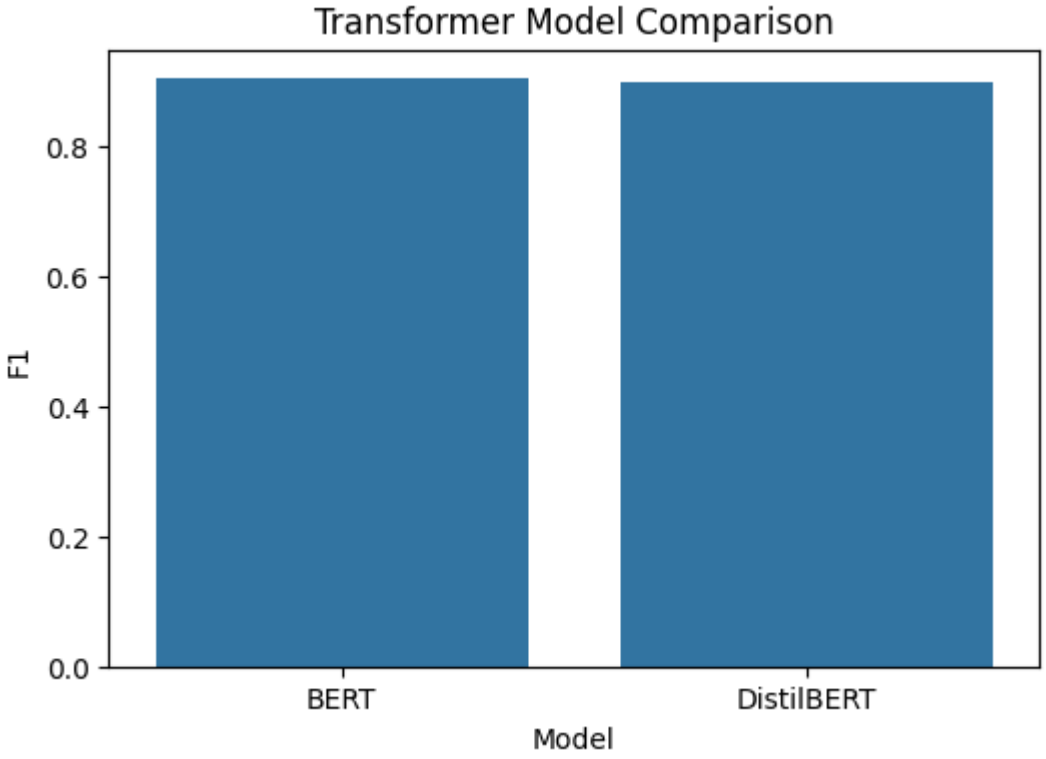
In [45]:
```
transformer_results = transformer_results.sort_values(
    by="F1",
    ascending=False
)

transformer_results
```

Out[45]:

	Model	Accuracy	Precision	Recall	F1
1	BERT	0.975484	0.898990	0.908163	0.903553
0	DistilBERT	0.972903	0.846847	0.959184	0.899522

In [46]:
```
transformer_results.to_csv(
    "../results/transformer_results.csv",
    index=False
)
```

In [47]:
```
plt.figure(figsize=(6,4))

sns.barplot(
    data=transformer_results,
    x="Model",
    y="F1"
)

plt.title(
    "Transformer Model Comparison"
)

plt.savefig(
    "../results/figures/transformer_comparison.png",
    bbox_inches="tight"
)

plt.show()
```



Conclusion

Both transformer models achieved strong performance on the SMS spam classification task.

Summary of Results:

Model	Accuracy	Precision	Recall	F1 Score
BERT	97.55%	89.90%	90.82%	90.36%
DistilBERT	97.29%	84.68%	95.92%	89.95%

BERT achieved the highest overall transformer performance and provided a better balance between precision and recall.

However, when compared with earlier experiments, transformer models did not outperform the best traditional machine learning model (Linear SVM) or the optimized LSTM model. This outcome suggests that for relatively small and highly structured datasets such as SMS spam detection, simpler models can remain highly competitive despite the increased representational power of transformer architectures.

The final notebook will combine all traditional machine learning, deep learning, and transformer results into a unified comparison framework.

Final Model Comparison and Selection

This notebook combines the results obtained from all previously implemented machine learning, deep learning, and transformer-based models for SMS Spam Classification.

The objective of this notebook is to:

- Consolidate performance metrics from all experiments
- Compare models using Accuracy, Precision, Recall, and F1 Score
- Rank models according to classification performance
- Identify the best-performing model
- Provide final conclusions regarding model suitability for SMS spam detection

The models evaluated include:

1. Logistic Regression
2. Naive Bayes
3. Linear SVM
4. LSTM
5. BiLSTM
6. DistilBERT
7. BERT

In [1]:
```
import pandas as pd
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt
```

Loading Experimental Results

The performance results generated from previous notebooks are loaded into this notebook.

The results are divided into three categories:

- Traditional Machine Learning Models
- Deep Learning Models
- Transformer-Based Models

Each result file contains Accuracy, Precision, Recall, and F1 Score obtained on the test dataset.

In [2]:
```
traditional_results = pd.read_csv(
    "../results/traditional_ml_results.csv"
)

traditional_results
```

Out[2]:

	Model	Accuracy	Precision	Recall	F1
0	Linear SVM	0.983226	0.967033	0.897959	0.931217
1	Naive Bayes	0.963871	0.986111	0.724490	0.835294
2	Logistic Regression	0.948387	1.000000	0.591837	0.743590

In [3]:
```
deep_learning_results = pd.read_csv(
    "../results/deep_learning_results.csv"
)

deep_learning_results
```

Out[3]:

	Model	Accuracy	Precision	Recall	F1
0	LSTM	0.980645	0.956044	0.887755	0.920635
1	BiLSTM	0.972903	0.952941	0.826531	0.885246

In [4]:
```
transformer_results = pd.read_csv(
    "../results/transformer_results.csv"
)

transformer_results
```

Out[4]:

	Model	Accuracy	Precision	Recall	F1
0	BERT	0.975484	0.898990	0.908163	0.903553
1	DistilBERT	0.972903	0.846847	0.959184	0.899522

In [5]:
```
all_results = pd.concat(

    [

        traditional_results,

        deep_learning_results,

        transformer_results

    ],

    ignore_index=True

)

all_results
```

Out[5]:

	Model	Accuracy	Precision	Recall	F1
0	Linear SVM	0.983226	0.967033	0.897959	0.931217
1	Naive Bayes	0.963871	0.986111	0.724490	0.835294
2	Logistic Regression	0.948387	1.000000	0.591837	0.743590
3	LSTM	0.980645	0.956044	0.887755	0.920635
4	BiLSTM	0.972903	0.952941	0.826531	0.885246
5	BERT	0.975484	0.898990	0.908163	0.903553
6	DistilBERT	0.972903	0.846847	0.959184	0.899522

In [6]:
```
all_results = all_results.sort_values(
    by="F1",
    ascending=False
)
all_results
```

Out[6]:

	Model	Accuracy	Precision	Recall	F1
0	Linear SVM	0.983226	0.967033	0.897959	0.931217
3	LSTM	0.980645	0.956044	0.887755	0.920635
5	BERT	0.975484	0.898990	0.908163	0.903553
6	DistilBERT	0.972903	0.846847	0.959184	0.899522
4	BiLSTM	0.972903	0.952941	0.826531	0.885246
1	Naive Bayes	0.963871	0.986111	0.724490	0.835294
2	Logistic Regression	0.948387	1.000000	0.591837	0.743590

Ranked Performance Results

The models have been sorted in descending order of F1 Score.

Higher F1 Scores indicate better balance between spam detection capability (Recall) and prediction reliability (Precision).

The ranking allows direct comparison of model effectiveness on the SMS Spam Collection dataset.

In [7]:
```
all_results.to_csv(
    "../results/final_model_comparison.csv",
    index=False
```

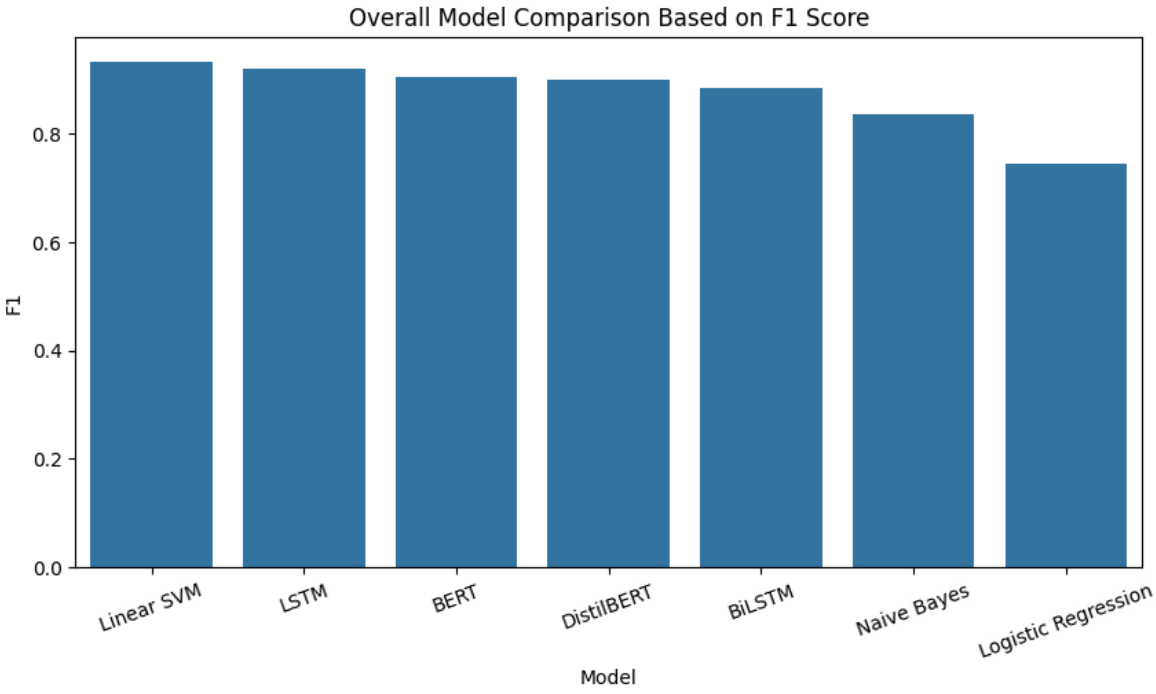
```
)
```

Comparison Based on F1 Score

This visualization compares all models using F1 Score.

Since F1 Score combines both Precision and Recall, it serves as the primary evaluation metric for identifying the most suitable spam classification model.

In [8]:
```
plt.figure(figsize=(10,5))

sns.barplot(

    data=all_results,

    x="Model",

    y="F1"

)

plt.title(

    "Overall Model Comparison Based on F1 Score"

)

plt.xticks(rotation=20)

plt.savefig(

    "../results/figures/final_model_comparison.png",

    bbox_inches="tight"

)

plt.show()
```



Comparison Based on Accuracy

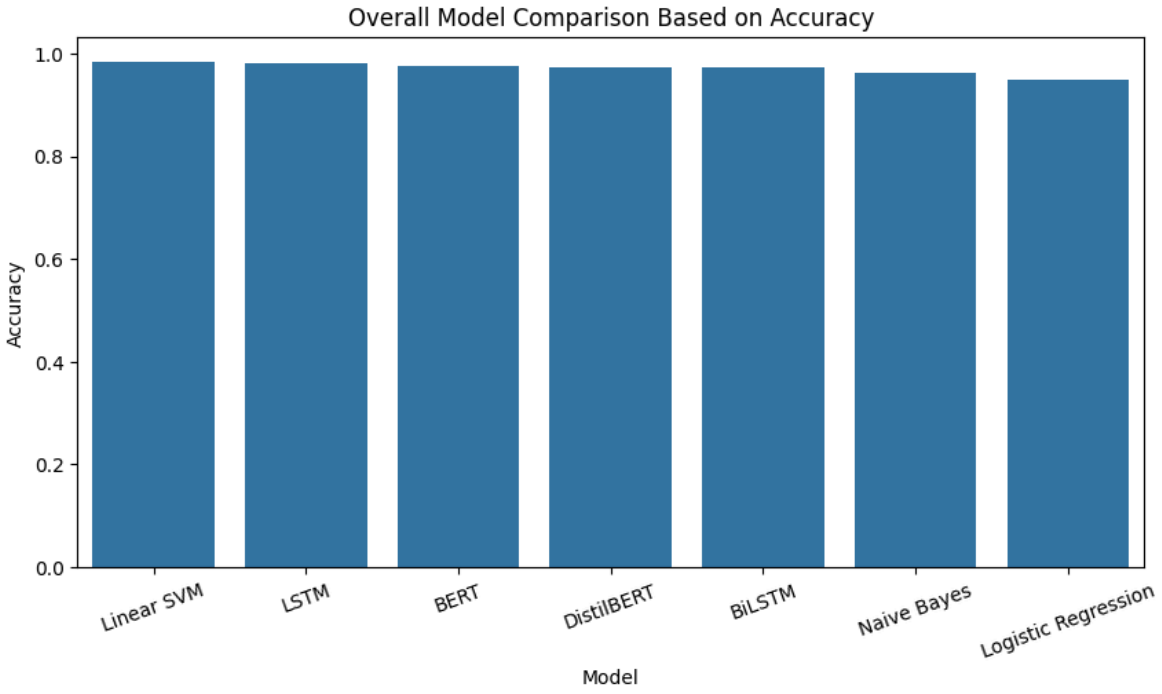
This visualization compares models using Accuracy.

Although most models achieve very high accuracy values, Accuracy alone may not adequately represent performance on imbalanced datasets.

Therefore, Accuracy is analyzed alongside Precision, Recall, and F1 Score.

In [9]:
```
plt.figure(figsize=(10,5))

sns.barplot(

    data=all_results,

    x="Model",

    y="Accuracy"

)

plt.title(

    "Overall Model Comparison Based on Accuracy"

)

plt.xticks(rotation=20)

plt.savefig(

    "../results/figures/final_accuracy_comparison.png",

    bbox_inches="tight"

)
```

```
plt.show()
```



Best Model Identification

The highest-ranked model is selected using F1 Score.

This model represents the best trade-off between detecting spam messages and minimizing false alarms.

The selected model can be considered the most suitable candidate for deployment in real-world SMS spam filtering applications.

In [10]:
```
top_model = all_results.iloc[0]

print("Best Model:")
print(top_model)
```

```
Best Model:
Model        Linear SVM
Accuracy       0.983226
Precision      0.967033
Recall         0.897959
F1             0.931217
Name: 0, dtype: object
```

Discussion of Results

Several interesting observations can be made from the experimental results:

Linear SVM

Linear SVM achieved the highest F1 Score of 0.9312 and emerged as the best-performing model.

LSTM

LSTM achieved strong performance with an F1 Score of 0.9206, demonstrating the effectiveness of sequential deep learning architectures for text classification.

BERT

BERT achieved an F1 Score of 0.9036 and demonstrated strong contextual understanding capabilities.

DistilBERT

DistilBERT achieved competitive performance while requiring fewer computational resources than BERT.

BiLSTM

BiLSTM performed slightly worse than LSTM despite having access to both forward and backward contextual information.

Naive Bayes

Naive Bayes delivered reasonable performance despite its simplicity and low computational cost.

Logistic Regression

Logistic Regression achieved the lowest F1 Score due to relatively poor recall performance on spam messages.

In [12]:
```
all_results
all_results.to_csv(
    "../results/final_model_comparison.csv",
    index=False
)
```

In [13]:
```
plt.figure(figsize=(10,4))

plt.axis("off")

table = plt.table(
    cellText=all_results.round(4).values,
    colLabels=all_results.columns,
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(10)
```

```
table.scale(1.2, 1.5)

plt.savefig(
    "../results/figures/final_results_table.png",
    bbox_inches="tight"
)

plt.show()
```

Model	Accuracy	Precision	Recall	F1
Linear SVM	0.9832	0.967	0.898	0.9312
LSTM	0.9806	0.956	0.8878	0.9206
BERT	0.9755	0.899	0.9082	0.9036
DistilBERT	0.9729	0.8468	0.9592	0.8995
BiLSTM	0.9729	0.9529	0.8265	0.8852
Naive Bayes	0.9639	0.9861	0.7245	0.8353
Logistic Regression	0.9484	1.0	0.5918	0.7436

Final Conclusion

A total of seven models were implemented and evaluated for SMS Spam Classification.

The models included:

- Logistic Regression
- Naive Bayes
- Linear SVM
- LSTM
- BiLSTM
- DistilBERT
- BERT

Performance evaluation was conducted using Accuracy, Precision, Recall, and F1 Score.

The final ranking based on F1 Score was:

1. Linear SVM (0.9312)
2. LSTM (0.9206)
3. BERT (0.9036)
4. DistilBERT (0.8995)
5. BiLSTM (0.8852)
6. Naive Bayes (0.8353)
7. Logistic Regression (0.7436)

The experimental results indicate that model performance is strongly influenced by the characteristics of the dataset. The SMS Spam Collection dataset consists of relatively short text messages and contains a limited number of training samples compared to the large-scale corpora typically required for deep learning and transformer-based models to realize their full potential.

Although advanced architectures such as LSTM, BiLSTM, BERT, and DistilBERT were successfully implemented and evaluated, they did not provide a significant performance advantage over the traditional machine learning approaches in this specific setting. The combination of TF-IDF feature extraction and Linear Support Vector Machine (SVM) achieved the highest overall F1 Score while maintaining excellent accuracy, precision, and recall.

These findings suggest that for short-text spam classification tasks with moderate dataset sizes, well-engineered traditional machine learning models can remain highly effective and computationally efficient alternatives to more complex deep learning and transformer-based approaches.

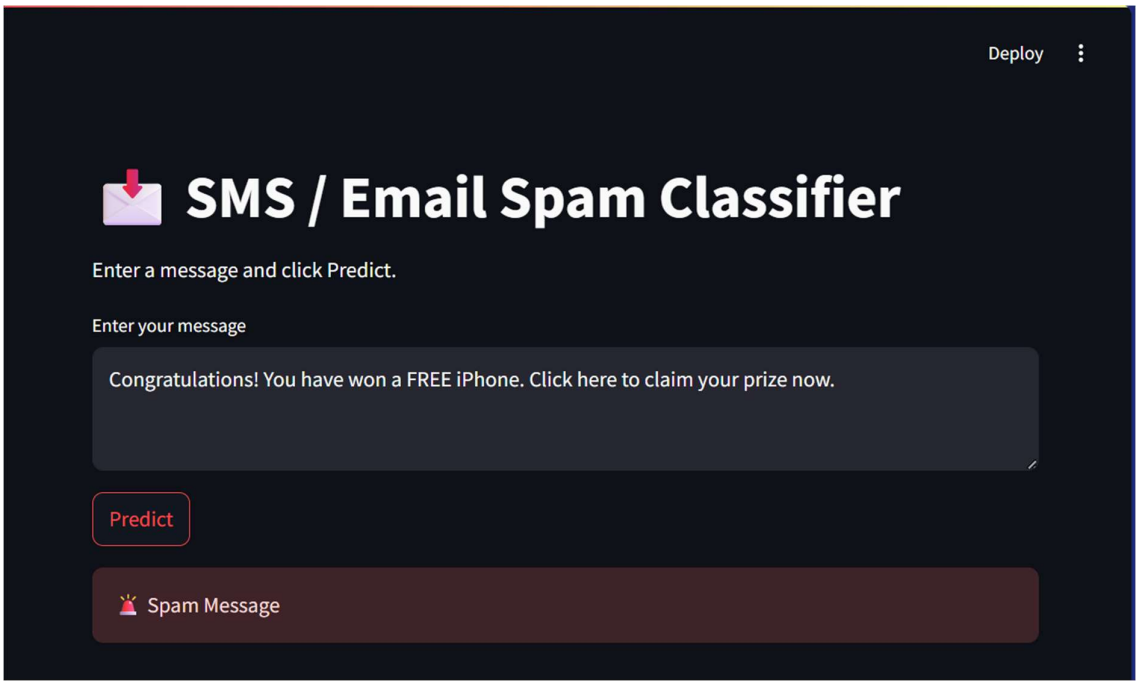
Based on the comparative evaluation of all seven models, Linear SVM is selected as the final best-performing model and is deployed through a Streamlit-based web application for real-time spam detection.

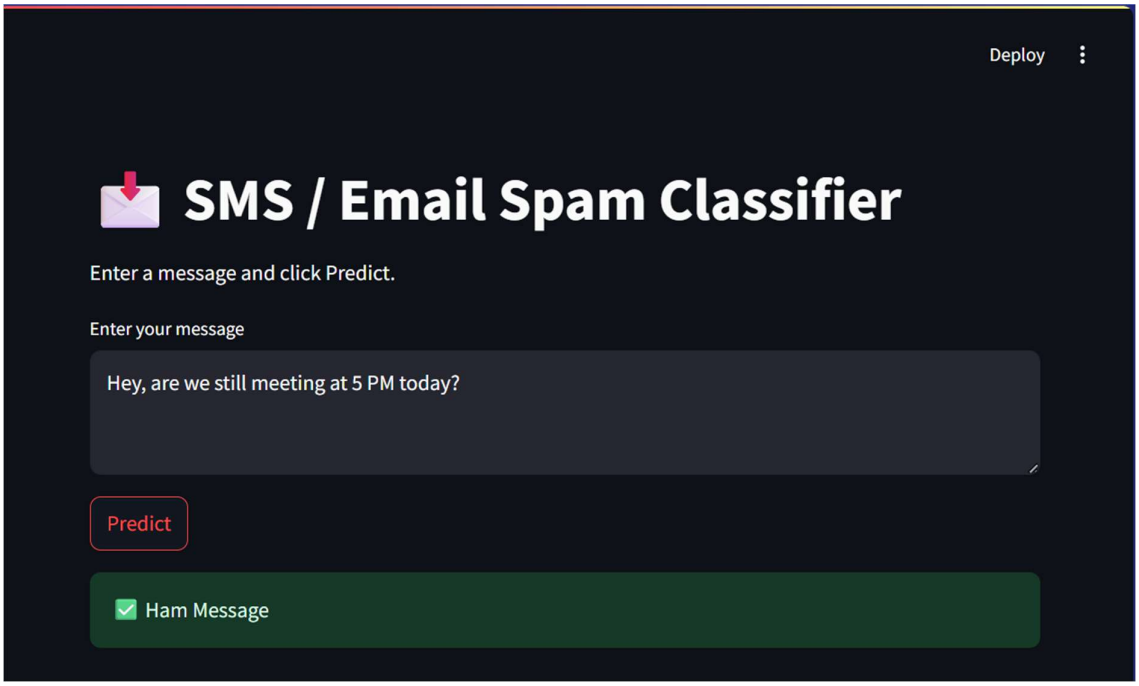
Model Deployment

A Streamlit-based web application was developed to demonstrate the real-time deployment of the Linear SVM spam detection model. The user enters a text message through the interface, which is preprocessed and transformed using the TF-IDF vectorizer. The trained Linear SVM model then predicts whether the message is spam or ham and displays the result instantly.

Code –

```
import streamlit as st
import joblib
model = joblib.load("linear_svm_model.pkl")
vectorizer = joblib.load("tfidf.pkl")
message = st.text_area("Enter a message")
if st.button("Predict"):
    msg = vectorizer.transform([message])
    pred = model.predict(msg)
    st.write(pred[0])
```


Deploy
SMS / Email Spam Classifier
Enter a message and click Predict.
Enter your message
Congratulations! You have won a FREE iPhone. Click here to claim your prize now.
Predict
Spam Message


Deploy
SMS / Email Spam Classifier
Enter a message and click Predict.
Enter your message
Hey, are we still meeting at 5 PM today?
Predict
Ham Message